PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2025

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

SentinelAI — Automated Fraud Detection & Behavioral Transaction Audit System

Submitted by:

SRN 1: PES1UG24AM129	Name 1 KARANAM SREEHITHA	5 C
SRN 2: PES1UG24AM134	Name 2 KHUSHI HIRENKUMAR PATEL	5 C

Class of Prof. ***[Raghu A.B.]***
[Rati D Joshi]

Abstract

SentinelAI is an Automated Fraud Detection and Behavioral Transaction Audit System developed to monitor financial transactions, identify potentially suspicious activities, and support the investigation of fraudulent transactions. With the increasing use of digital payment systems, monitoring large volumes of financial transactions and maintaining proper investigation records have become important requirements.

The proposed system provides a centralized web-based platform through which administrators and fraud analysts can monitor transactions, examine risk assessments, manage fraud alerts, and track investigation cases. It also maintains investigation notes and audit records to support transparency and traceability of important system activities.

The application follows a hybrid database architecture by integrating MySQL and MongoDB. MySQL is used to store structured information such as customer details, accounts, transactions, fraud alerts, investigation cases, and user records.

MongoDB is used to manage flexible document-oriented information, including transaction risk-analysis results, behavioral events, and investigation evidence. The application is developed using Next.js, React, and TypeScript, with backend API routes facilitating communication between the application and the databases. Authentication and role-based access control are incorporated to provide separate access privileges for administrators and fraud analysts.

The project demonstrates the practical application of database management concepts, including relational database operations, transaction management, SQL joins, MongoDB document operations, aggregation, and database–application integration.

Overall, SentinelAI aims to provide an organized and efficient platform for transaction monitoring, fraud alert management, and behavioral transaction auditing.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5-7
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	8-10
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	10-12
5.	SQL DDL Statements	12
6.	SQL DML Statements	13-15
7.	NoSQL/ Vector Database Commands	15-19
8.	Results a) Resulting Tables' Screenshots	20-26
9.	Next.js-Project Tree Structure	27-28
10.	Few sample Screenshots of application code and demo	29-33
11.	Evaluation Rubrics- Completion Status	34

1. Introduction

Problem Statement

The rapid growth of digital banking, online payments, and electronic financial transactions has increased the need for effective transaction monitoring and fraud detection mechanisms. Financial institutions process large volumes of transactions every day, making it difficult to manually identify suspicious activities, unusual transaction patterns, and potentially fraudulent behaviour.

A major challenge is not only identifying suspicious transactions but also maintaining proper records of fraud alerts, investigation activities, and user actions. Without a centralized system, monitoring transactions, tracking investigation progress, and maintaining historical audit information can become difficult.

Therefore, there is a need for an integrated system that can monitor financial transactions, evaluate their risk levels, generate and manage fraud alerts, and provide facilities for investigating suspicious activities. The system should also maintain structured transactional records and flexible behavioural analysis data while ensuring controlled access for different categories of users.

To address these challenges, SentinelAI – Automated Fraud Detection and Behavioral Transaction Audit System, is developed as a database-driven web application. It integrates MySQL and MongoDB to manage structured financial information and flexible analytical data, respectively. Through a centralized interface, administrators and fraud analysts can monitor transactions, review risk assessments, manage alerts, and maintain investigation records.

2.User Requirements Specification

2.1 Functional Requirements

The functional requirements describe the major operations and services provided by the SentinelAI application.

1. **User Authentication:** The system should allow registered users to log in using valid credentials.
2. **Role-Based Access Control:** The application should support different user roles, including Administrator and Fraud Analyst, with appropriate access privileges.
3. **Transaction Monitoring:** Authorized users should be able to view and monitor financial transaction records.
4. **Risk Assessment:** The system should retrieve and display risk scores, risk levels, and associated risk-analysis information for transactions.
5. **Fraud Alert Management:** The application should allow authorized users to review and manage alerts generated for potentially suspicious transactions.
6. **Investigation Management:** The system should provide facilities to create, assign, update, and track investigation cases.
7. **Investigation Notes:** Users should be able to maintain notes associated with investigation cases.
8. **Audit Logging:** Important application activities should be recorded to maintain a history of system operations.
9. **Database Integration:** The application should communicate with MySQL and MongoDB through backend API routes.
10. **Data Retrieval:** Transaction and investigation information should be accessible through appropriate retrieval and filtering mechanisms.

2.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes expected from the application.

1. **Security:** The application should protect sensitive information through authentication and role-based authorization.
2. **Reliability:** Database operations should maintain consistency and minimize the possibility of incomplete updates.
3. **Maintainability:** The application should follow a modular structure to simplify future development and maintenance.
4. **Usability:** The web interface should provide a convenient and understandable experience for administrators and fraud analysts.
5. **Scalability:** The database architecture should support the management of increasing volumes of structured and document-oriented information.
6. **Data Integrity:** Appropriate database relationships, constraints, and validation mechanisms should help maintain accurate and consistent records.

2.3 Technology Stack

The following technologies are used in the development of SentinelAI.

Component	Technology Used	Purpose
Frontend	Next.js and React	Development of the web-based user interface and dashboard.
Programming Language	TypeScript	Provides static typing and improves application maintainability.
Styling	Tailwind CSS	Used for designing and styling the application interface.
Backend	Next.js API Routes	Handles application requests and database communication.
Relational Database	MySQL	Stores structured information such as customers, accounts, transactions, users, and fraud alerts.
NoSQL Database	MongoDB	Stores flexible risk-analysis documents, behavioral events, and investigation evidence.
Authentication	NextAuth.js	Manages user authentication and sessions.
Password Security	bcryptjs	Supports password hashing and verification.
Input Validation	Zod	Provides schema-based validation for application inputs.
Runtime Environment	Node.js	Executes the server-side application.
Version Control	Git and GitHub	Maintains source code and supports collaborative development.

2.4 Justification for the Technology Stack

Next.js and React: Next.js is selected for developing the web application because it supports component-based development, application routing, and backend API functionality. React enables the development of reusable interface components, making it suitable for dashboards and transaction monitoring interfaces.
TypeScript: TypeScript is used to improve code reliability through static type checking. It helps identify potential programming errors during development and makes the application easier to maintain.
MySQL: MySQL is selected as the relational database because the application manages structured financial information with clearly defined relationships. Entities such as customers, accounts, transactions, users, fraud alerts, and investigation cases require relational data management. MySQL supports primary keys, foreign keys, constraints, joins, indexing, and transaction management, making it suitable for maintaining consistency among related records.

MongoDB: MongoDB is selected as the NoSQL database because risk-analysis results, behavioral events, and investigation evidence may contain flexible and varying attributes. Its document-

oriented structure allows this information to be stored in JSON-like documents, making it suitable for managing additional analytical information without requiring a rigid relational structure.
NextAuth.js: NextAuth.js is used to implement authentication and session management, allowing registered users to access the application through their credentials.
bcryptjs: bcryptjs is used to securely hash and verify user passwords rather than storing passwords directly in plain text.
Tailwind CSS: Tailwind CSS is used to maintain a consistent and responsive user interface with reusable styling utilities.
Git and GitHub: Git and GitHub are used for source-code version control, project collaboration, and maintaining the development history.

2.5 Justification for Using MySQL and MongoDB Together

SentinelAI requires the management of two different categories of information: structured transactional records and flexible analytical information.

MySQL is responsible for maintaining structured records and relationships between customers, accounts, transactions, users, alerts, and investigation cases. These records require relational integrity and reliable transaction processing.

MongoDB is responsible for storing additional information such as risk-analysis results, behavioral events, and investigation evidence. These documents may contain varying attributes depending on the type of analysis or evidence being recorded.

Using both databases allows the system to select an appropriate storage model based on the nature of the data. This hybrid database architecture demonstrates the practical application of relational and NoSQL database technologies within a single web application.

3.E-R Model

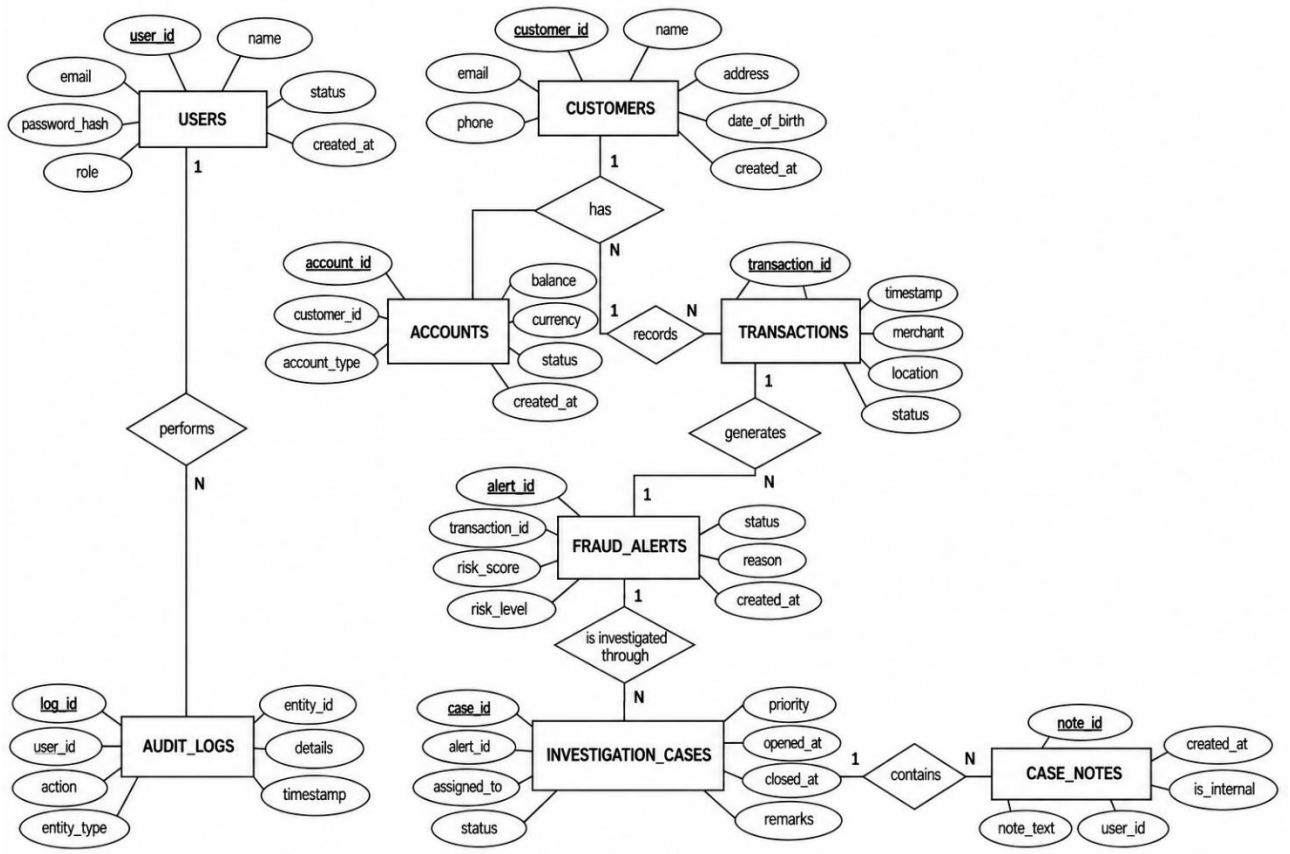
The Entity-Relationship (E-R) model represents the conceptual database structure of SentinelAI. It identifies the major entities involved in transaction monitoring, fraud detection, investigation management, and audit tracking, along with their relationships.

The system follows a hybrid database architecture consisting of MySQL and MongoDB. The primary relational entities are maintained in MySQL, while additional analytical and behavioral information is stored as documents in MongoDB.

The major entities identified in the system are:

Sl. No.	Entity	Description
1.	USERS	Stores user information, login details, and role-related information for administrators and fraud analysts.
2.	CUSTOMERS	Maintains customer information associated with financial transactions.
3.	ACCOUNTS	Stores financial account information associated with customers.
4.	TRANSACTIONS	Maintains transaction details such as transaction identifiers, amounts, timestamps, and associated account information.
5.	FRAUD_ALERTS	Stores alerts associated with potentially suspicious transactions and their review status.
6.	INVESTIGATION_CASES	Maintains information about investigations initiated for fraud-related alerts.
7.	CASE_NOTES	Stores notes and updates recorded during the investigation of a case.
8.	AUDIT_LOGS	Maintains records of important activities performed within the application.

E-R DIAGRAM :


user_id
name
email
password_hash
role
status
created_at
USERS
customer_id
name
email
phone
address
date_of_birth
created_at
CUSTOMERS
has
account_id
customer_id
account_type
balance
currency
status
created_at
ACCOUNTS
records
transaction_id
timestamp
merchant
location
status
TRANSACTIONS
performs
generates
alert_id
transaction_id
risk_score
risk_level
status
reason
created_at
FRAUD_ALERTS
is investigated through
log_id
user_id
action
entity_type
entity_id
details
timestamp
AUDIT_LOGS
case_id
alert_id
assigned_to
status
priority
opened_at
closed_at
remarks
INVESTIGATION_CASES
contains
note_id
note_text
user_id
created_at
is_internal
CASE_NOTES
1
N

Additional MongoDB Collections

Apart from the relational entities, the system uses MongoDB collections to store flexible document-oriented information.

Sl. No.	Collection	Description
1.	risk_analysis	Stores transaction-related risk assessment information, including risk scores and risk levels.
2.	behavior_events	Stores behavioral event information associated with transaction monitoring.
3.	investigation_evidence	Stores document-oriented evidence associated with fraud investigations.

These collections complement the relational database by storing flexible analytical and investigation-related information.

4.Relational Model

4.1 List of Relations
The relational model of SentinelAI consists of a set of interconnected relations designed to manage users, customers, financial accounts, transactions, fraud alerts, investigation cases, and audit information.
The major relations identified from the application's SQL queries are:

Sl. No.	Relation	Purpose
1.	USERS	Stores user credentials, names, roles, and account status.
2.	CUSTOMERS	Stores customer-related information.
3.	ACCOUNTS	Maintains financial account information associated with customers.
4.	TRANSACTIONS	Stores financial transaction records.
5.	FRAUD_ALERTS	Maintains alerts generated for potentially suspicious transactions.
6.	INVESTIGATION_CASES	Stores investigation details associated with fraud alerts.
7.	CASE_NOTES	Maintains notes recorded during investigations.
8.	AUDIT_LOGS	Records important activities performed by application users.

4.2 Relations That Are Not Entities in the E-R Model
Apart from the base relations, the application also uses database views to retrieve information from multiple related tables.

1. v_transaction_monitoring
This view is referenced by the transaction monitoring, transaction audit, and risk-analysis modules. It provides a consolidated representation of transaction information, including:

- Transaction ID
- Transaction reference
- Customer ID and customer name
- Account number
- Transaction type
- Transaction amount
- Merchant
- Transaction location and status
- Transaction timestamp
- Average transaction amount
- Maximum normal transaction amount
- Customer's usual location

The view simplifies transaction retrieval by presenting related information through a single database object.

2. v_fraud_alert_dashboard

This view is mentioned in the project documentation. Its actual definition and availability should be verified in the MySQL database before including it as an implemented view.

Relational Schema
The relational schema represents the database relations and their logical connections.
The following attributes are identifiable from the application's SQL queries. PK and FK markings must be confirmed against the actual MySQL table definitions.

USERS
USERS(user_id, full_name, email, password_hash, role, is_active)
CUSTOMERS
CUSTOMERS(customer_id, full_name)
ACCOUNTS
ACCOUNTS(account_id, customer_id)
TRANSACTIONS
TRANSACTIONS(transaction_id, transaction_reference, account_id, transaction_type, amount, merchant, transaction_location, transaction_status, transaction_time, avg_transaction_amount, max_normal_amount, usual_location)
FRAUD_ALERTS
FRAUD_ALERTS(alert_id, transaction_id, assigned_to, risk_score, risk_level, alert_reason, alert_status, created_at, resolved_at)
INVESTIGATION_CASES
INVESTIGATION_CASES(case_id, alert_id, assigned_to, case_status, priority, conclusion, opened_at, closed_at)
CASE_NOTES
CASE_NOTES(note_id, case_id, author_id, note_text, created_at)
AUDIT_LOGS

AUDIT_LOGS(log_id, user_id, action_type, entity_type, entity_id, details, created_at)
The major logical relationships include:

- A customer can be associated with multiple accounts.
- An account can be associated with multiple transactions.
- A transaction can be associated with fraud alerts.
- A fraud alert can be associated with investigation cases.
- An investigation case can contain multiple notes.
- A user can be associated with multiple audit records.
- Users can be assigned investigation responsibilities and author investigation notes.

The primary keys, foreign keys, and exact relationship cardinalities are to be finalized using the actual MySQL schema.

5.SQL DDL Statements

Data Definition Language (DDL) is used to define and manage the structure of relational database objects. In SentinelAI, MySQL is used as the primary relational database for maintaining structured financial and investigation-related information.

The database design consists of tables representing users, customers, accounts, transactions, fraud alerts, investigation cases, case notes, audit logs, and behavioral profiles.

The major DDL concepts relevant to the project include:

1. **CREATE:** Used to create database tables and views.
2. **ALTER:** Used to modify existing table structures.
3. **DROP:** Used to remove database objects when required.
4. **PRIMARY KEY:** Ensures unique identification of records.
5. **FOREIGN KEY:** Establishes relationships between related tables.
6. **NOT NULL:** Ensures that mandatory attributes contain values.
7. **UNIQUE:** Prevents duplicate values in specified attributes.
8. **CHECK:** Enforces specified data restrictions.
9. **INDEX:** Supports efficient retrieval of records.
10. **VIEW:** Provides a reusable representation of data retrieved from one or more tables.

Database Views

The project documentation identifies two monitoring views:

- v_transaction_monitoring
- v_fraud_alert_dashboard

The v_transaction_monitoring view is used by the application and MongoDB seeding script to retrieve consolidated transaction information.
The database views simplify data retrieval and provide a structured interface for application queries.

The actual SQL statements used to create tables, define primary and foreign keys, configure constraints, and create views should be included in this section.
The statements should be accompanied by their execution results from MySQL Workbench or the corresponding SQL execution environment.

6.SQL DML Statements

Data Manipulation Language (DML) consists of SQL commands used to retrieve, insert, update, and delete records stored in database tables.
SentinelAI uses MySQL to manage transaction information, fraud alerts, investigation cases, investigation notes, and audit records.
The application communicates with MySQL using the MySQL2 library. Parameterized queries are used to pass values securely to SQL statements.

6.1 SELECT Operation
The MongoDB seeding script retrieves transaction information from the MySQL v_transaction_monitoring view.

```
SELECT
    transaction_id,
    transaction_reference,
    customer_id,
    customer_name,
    amount,
    transaction_location,
    transaction_status,
    transaction_time,
    max_normal_amount,
    usual_location
FROM v_transaction_monitoring
ORDER BY transaction_id;
```

Explanation: Retrieves transaction details from the monitoring view. These records are subsequently used for risk assessment and behavioral event generation.

JOIN Operation
The seeding script also retrieves investigation information by joining multiple relational tables.

```
SELECT
    ic.case_id,
    ic.alert_id,
    ic.case_status,
    ic.priority,
    fa.transaction_id,
    fa.risk_score,
    fa.risk_level,
    fa.alert_status,
    t.transaction_reference,
    t.amount,
    t.transaction_location,
    t.transaction_status
```

```
FROM investigation_cases ic
INNER JOIN fraud_alerts fa
   ON fa.alert_id = ic.alert_id
INNER JOIN transactions t
   ON t.transaction_id = fa.transaction_id
ORDER BY ic.case_id;
```

Explanation: Combines investigation cases, fraud alerts, and transaction information to generate investigation evidence documents in MongoDB.

INSERT Operation

The application supports creating investigation records and recording investigation notes.
An illustrative SQL statement for inserting an investigation note is:

```
INSERT INTO case_notes
   (case_id, author_id, note_text)
```

Explanation: Adds an investigation note associated with a particular case and its author.

UPDATE Operation

The application supports updating fraud-alert and investigation-related information.
An illustrative SQL statement for updating an alert status is:

```
UPDATE fraud_alerts
SET alert_status
WHERE alert_id
```

Explanation: Updates the status of a selected fraud alert based on the value supplied by an authorized user.

DELETE Operation

The DELETE operation can be used to remove records when permitted by the database design and application requirements.
A representative SQL statement is:

```
DELETE FROM case_notes
WHERE note_id
```

This statement illustrates the deletion of a selected investigation note. It should only be included as an implemented project operation if the application actually supports it.

Transaction Management

Transaction management helps maintain consistency when multiple related database operations need to be performed together.
The following SQL commands represent the basic transaction-control operations:

```
START TRANSACTION;


COMMIT;
```

If an operation fails, the transaction can be rolled back:

```
ROLLBACK;
```

These commands help prevent partially completed database operations when used within an appropriate transaction-management implementation.

Summary

The SQL DML operations support the retrieval and management of structured records in SentinelAI. SELECT and JOIN queries retrieve information from related tables, while INSERT and UPDATE operations support investigation and fraud-alert management. Parameterized queries help maintain secure communication between the Next.js backend and the MySQL database.

7.NoSQL/ Vector Database Commands

MongoDB Database Implementation

SentinelAI uses MongoDB as its NoSQL database to store flexible, document-oriented information related to transaction risk analysis, customer behavioral events, and investigation evidence.
The MongoDB database is named fraud_detection_db. It contains three major collections:

Sl. No.	Collection	Purpose
1.	risk_analysis	Stores risk scores, risk levels, reasons for risk classification, and transaction analysis information.
2.	behavior_events	Stores behavioral events identified from transaction information, including unusual amounts, location mismatches, and transaction status.
3.	investigation_evidence	Stores supporting investigation records associated with fraud alerts and investigation cases.

MongoDB Connection

The application uses the MongoDB Node.js driver to establish a connection with the database. The connection URI is obtained through an environment variable.

```
const { MongoClient } = require("mongodb");

const mongo = new MongoClient(process.env.MONGODB_URI);

await mongo.connect();

const database = mongo.db("fraud_detection_db");
```

The three collections are accessed as follows:

```
const riskCollection =
   database.collection("risk_analysis");

const behaviorCollection =
   database.collection("behavior_events");

const evidenceCollection =
   database.collection("investigation_evidence");
```

Collection Structures

A. risk_analysis

The risk_analysis collection stores the results of transaction risk assessment.
The documents contain the following attributes:

- transactionId
- transactionReference
- customerId
- customerName
- amount
- location
- transactionStatus
- riskScore
- riskLevel
- reasons
- dataQualityIssue
- analyzedAt

An example document is:

```
{
   transactionId: 101,
   transactionReference: "TXN-101",
   customerId: 10,
   amount: 25000,
   location: "Bengaluru",
   transactionStatus: "SUCCESS",
   riskScore: 75,
   riskLevel: "HIGH",
   reasons: [
      "Transaction amount exceeds normal spending limit"
   ],
   dataQualityIssue: false,
   analyzedAt: new Date()
}
```

The above is an illustrative document structure.

B. behavior_events

The behavior_events collection maintains behavioral events derived from transaction records.
Its document attributes include:

- eventId
- transactionId
- transactionReference
- customerId
- customerName
- eventType
- severity
- description
- amount
- location
- usualLocation

- transactionStatus
- eventTime
- syntheticDemoData
- createdAt

The system identifies different event categories, including:

1. BLOCKED_TRANSACTION
2. FAILED_TRANSACTION
3. AMOUNT_DEVIATION
4. LOCATION_MISMATCH
5. TRANSACTION_ACTIVITY

These events help represent transaction behaviour and identify unusual activities.

C. investigation_evidence

The investigation_evidence collection stores synthetic investigation-related evidence documents linked to existing investigation cases.

Its document attributes include:

- evidenceId
- caseId
- alertId
- transactionId
- evidenceType
- title
- description
- details
- source
- syntheticDemoData
- recordedAt

The details field contains additional information such as case status, case priority, alert status, risk level, risk score, transaction reference, transaction status, amount, and location.

The collection supports flexible storage of investigation-related information in document format.

Rule-Based Transaction Risk Analysis

The application implements a rule-based function named analyzeTransaction().

It evaluates the transaction amount against the customer's normal spending limit and compares the transaction location with the customer's usual location.

The risk-scoring rules are:

Condition	Risk Score Added
Amount exceeds normal spending limit	15
Amount is at least twice the normal limit	25
Amount is at least three times the normal limit	35
Amount is at least five times the normal limit	50
Transaction location differs from usual location	30

The final risk score is capped at 100.

The risk categories are assigned as follows:

Risk Score	Risk Level
0–39	LOW
40–69	MEDIUM
70–100	HIGH

The system also maintains a data-quality indicator when important information, such as transaction amount or location, is missing or invalid.

8.MongoDB Data Manipulation Operations

A. Bulk Write Operation

The application uses bulkWrite() to process transaction risk-analysis documents in batches of 500.

```
await riskCollection.bulkWrite(
   operations,
   { ordered: true }
);
```

This allows multiple database operations to be submitted together, reducing the overhead of individual database requests.

Update Operation with Upsert

The application uses updateOne() with the upsert option to insert new documents or update existing ones.

```
{
   updateOne: {
      filter: {
         transactionId: document.transactionId
      },
      update: {
         $set: document
      },
      upsert: true
   }
}
```

If a matching transaction document exists, its information is updated. Otherwise, a new document is created.

The same approach is used for behavioral events and investigation evidence, with their respective identifiers.

Count Documents

The application uses countDocuments() to retrieve the number of documents in each collection.

```
const total = await riskCollection.countDocuments();

console.log(total);
```

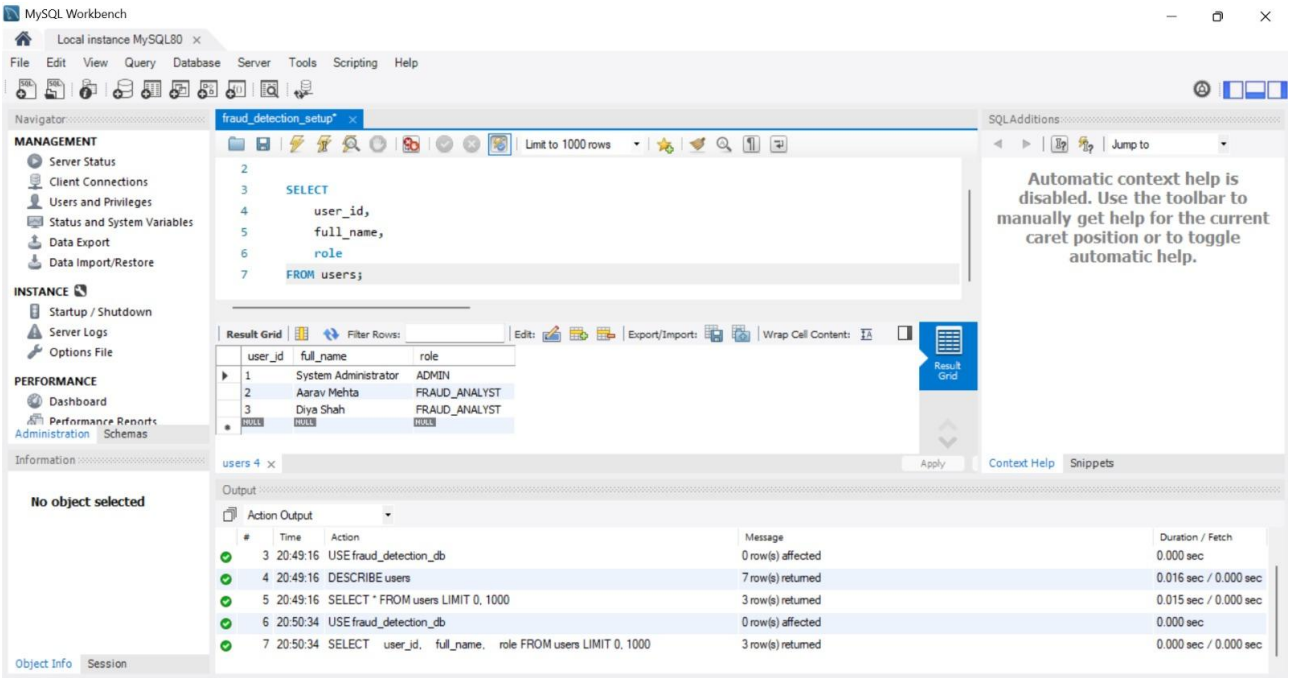
This helps verify the number of records populated during the seeding process.

MongoDB Aggregation Pipeline

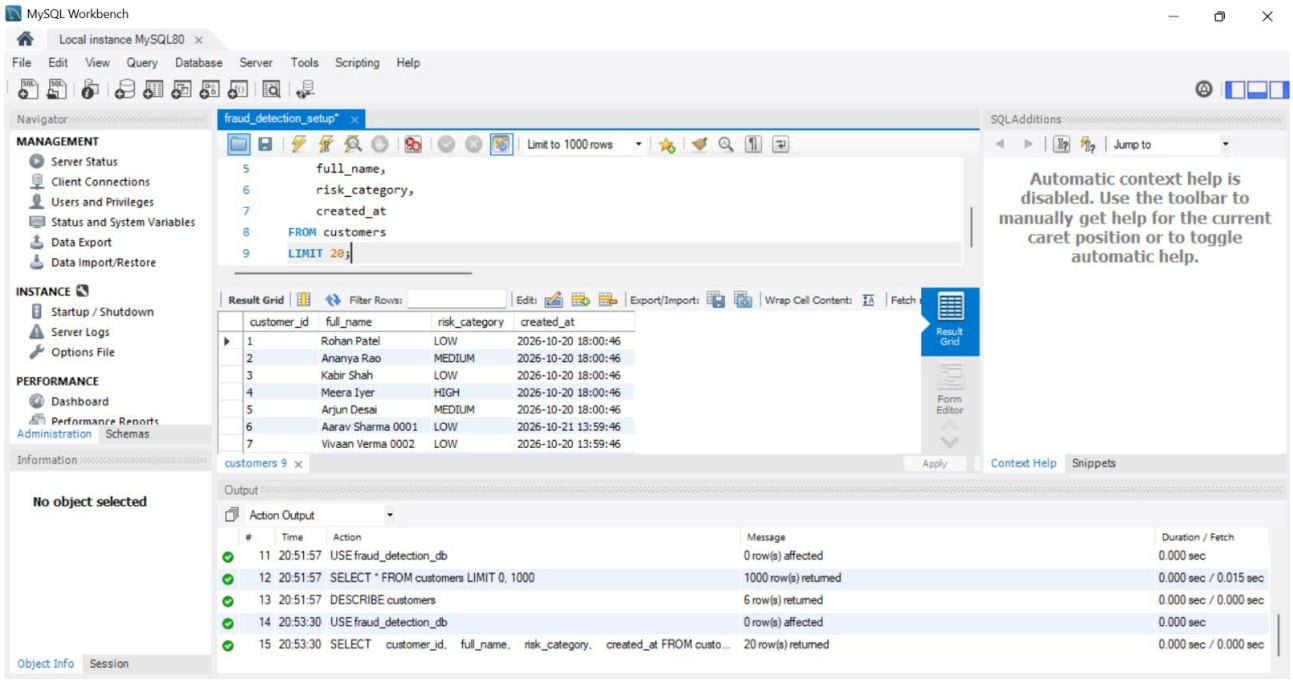
SentinelAI also demonstrates the use of MongoDB aggregation to retrieve the latest risk-analysis document for each transaction.
The transaction audit API uses the following aggregation stages:
1. match: Filters documents using the selected transaction identifiers.
2. sort: Sorts analysis records by timestamp in descending order.
3. group: Groups records based on transaction identifiers.
4. first: Selects the latest analysis document from each group.
This enables the application to retrieve the most recent risk assessment associated with a transaction.

Integration Between MySQL and MongoDB
The project integrates MySQL and MongoDB to manage different categories of information. MySQL provides the structured transaction and investigation records, while MongoDB stores flexible risk-analysis results, behavioral events, and investigation evidence.
The MongoDB seeding script retrieves transaction records from the MySQL v_transaction_monitoring view and applies the risk-assessment logic before storing the results in MongoDB.
For investigation evidence, the script retrieves information by joining investigation_cases, fraud_alerts, and transactions in MySQL.
This integration demonstrates how relational and document-oriented databases can work together within a single application.

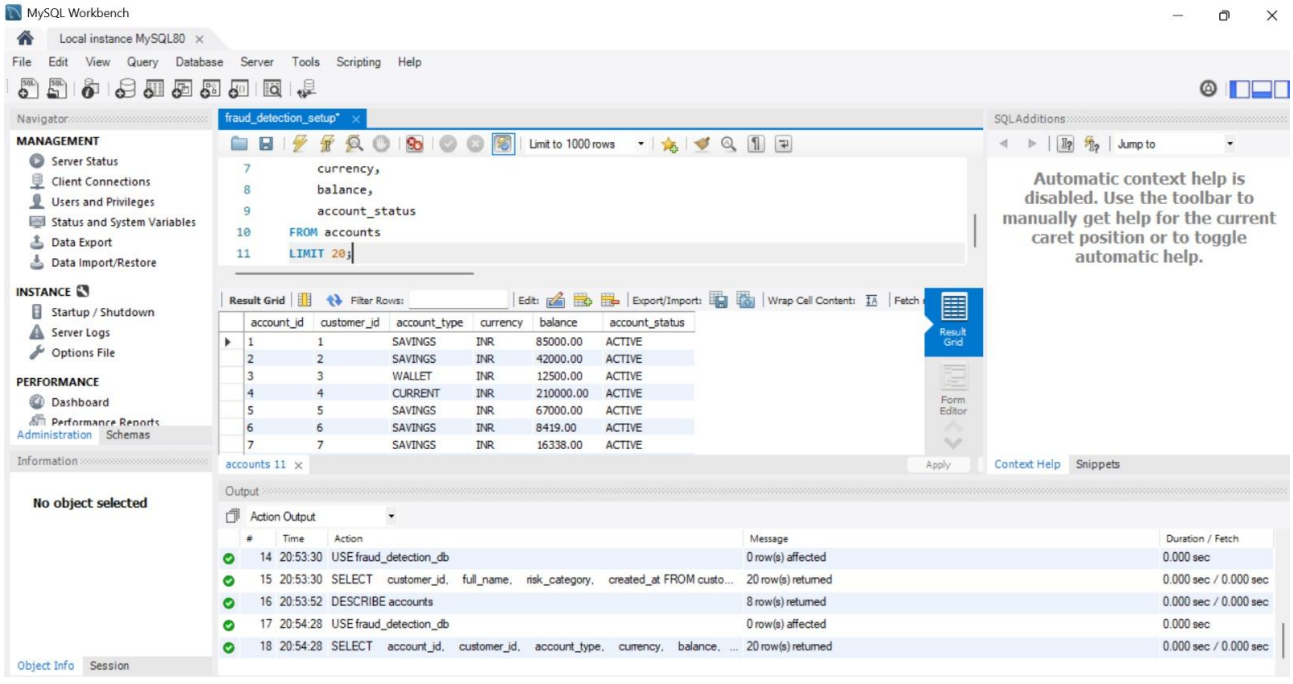
1.



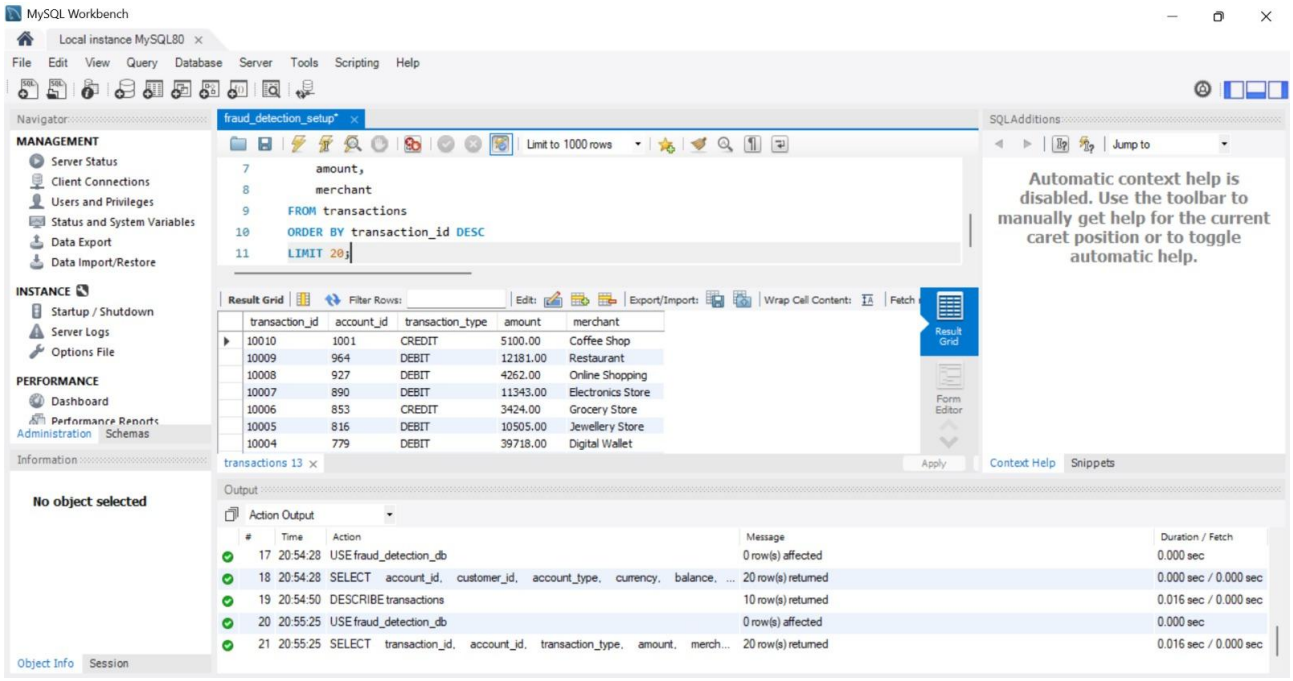
2


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
full_name,
risk_category,
created_at
FROM customers
LIMIT 20;
Result Grid
customer_id full_name risk_category created_at
1 Rohan Patel LOW 2026-10-20 18:00:46
2 Ananya Rao MEDIUM 2026-10-20 18:00:46
3 Kabir Shah LOW 2026-10-20 18:00:46
4 Meera Iyer HIGH 2026-10-20 18:00:46
5 Arjun Desai MEDIUM 2026-10-20 18:00:46
6 Aarav Sharma 0001 LOW 2026-10-21 13:59:46
7 Vivaan Verma 0002 LOW 2026-10-20 13:59:46
customers 9
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
11 20:51:57 USE fraud_detection_db 0 row(s) affected 0.000 sec
12 20:51:57 SELECT * FROM customers LIMIT 0, 1000 1000 row(s) returned 0.000 sec / 0.015 sec
13 20:51:57 DESCRIBE customers 6 row(s) returned 0.000 sec / 0.000 sec
14 20:53:30 USE fraud_detection_db 0 row(s) affected 0.000 sec
15 20:53:30 SELECT customer_id, full_name, risk_category, created_at FROM custo... 20 row(s) returned 0.000 sec / 0.000 sec

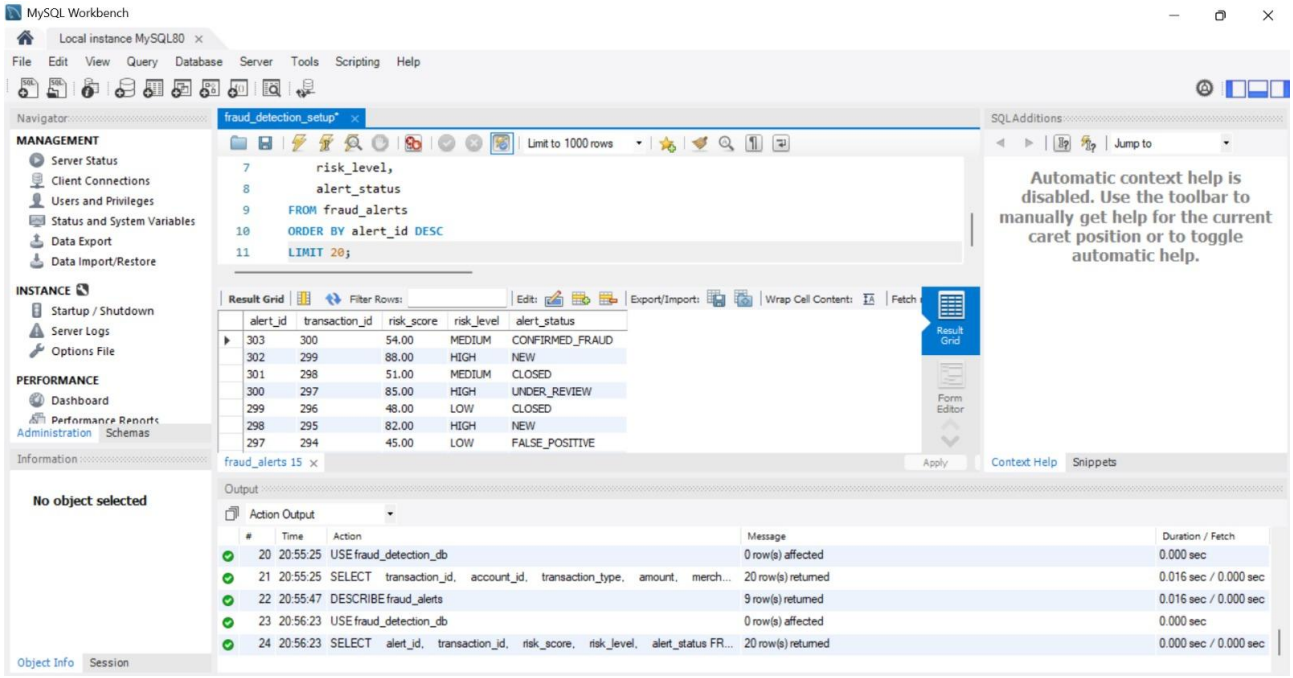
3.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
currency,
balance,
account_status
FROM accounts
LIMIT 20;
Result Grid
account_id customer_id account_type currency balance account_status
1 1 SAVINGS INR 85000.00 ACTIVE
2 2 SAVINGS INR 42000.00 ACTIVE
3 3 WALLET INR 12500.00 ACTIVE
4 4 CURRENT INR 210000.00 ACTIVE
5 5 SAVINGS INR 67000.00 ACTIVE
6 6 SAVINGS INR 8419.00 ACTIVE
7 7 SAVINGS INR 16338.00 ACTIVE
accounts 11
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
14 20:53:30 USE fraud_detection_db 0 row(s) affected 0.000 sec
15 20:53:30 SELECT customer_id, full_name, risk_category, created_at FROM custo... 20 row(s) returned 0.000 sec / 0.000 sec
16 20:53:52 DESCRIBE accounts 8 row(s) returned 0.000 sec / 0.000 sec
17 20:54:28 USE fraud_detection_db 0 row(s) affected 0.000 sec
18 20:54:28 SELECT account_id, customer_id, account_type, currency, balance, ... 20 row(s) returned 0.000 sec / 0.000 sec

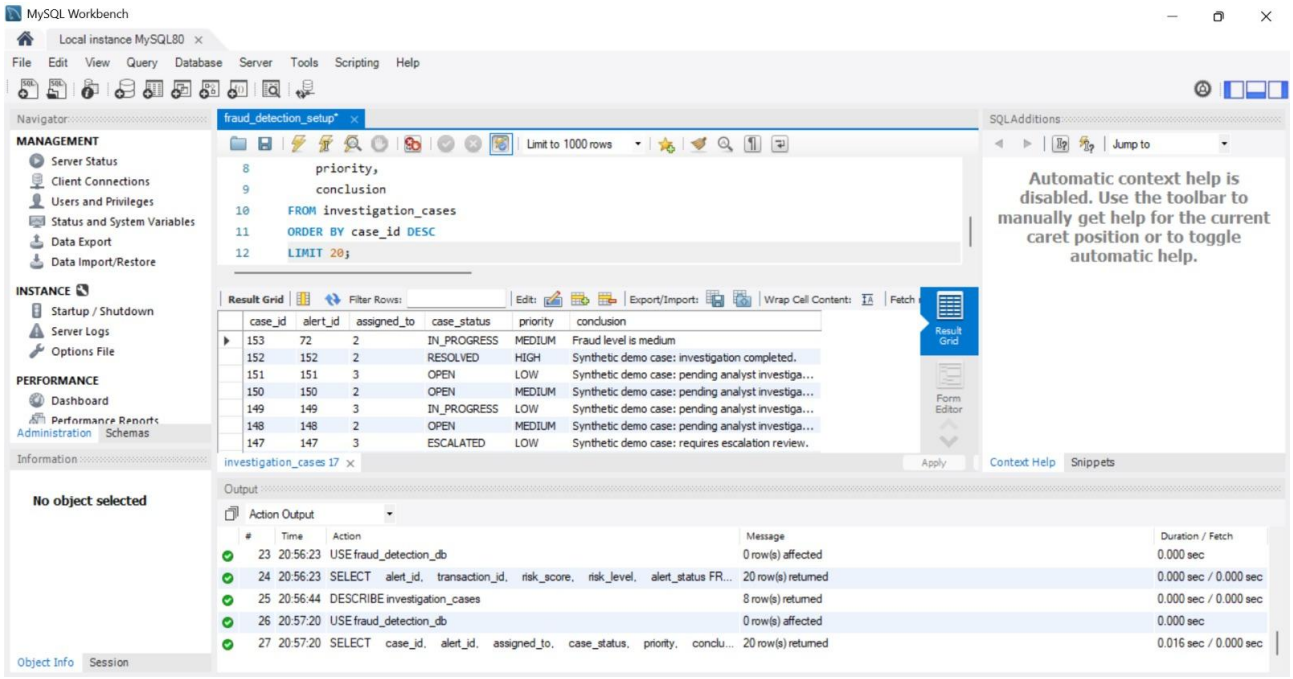
4.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
7 amount,
8 merchant
9 FROM transactions
10 ORDER BY transaction_id DESC
11 LIMIT 20;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch
transaction_id account_id transaction_type amount merchant
10010 1001 CREDIT 5100.00 Coffee Shop
10009 964 DEBIT 12181.00 Restaurant
10008 927 DEBIT 4262.00 Online Shopping
10007 890 DEBIT 11343.00 Electronics Store
10006 853 CREDIT 3424.00 Grocery Store
10005 816 DEBIT 10505.00 Jewellery Store
10004 779 DEBIT 39718.00 Digital Wallet
transactions 13
Result Grid
Form Editor
Apply
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
17 20:54:28 USE fraud_detection_db 0 row(s) affected 0.000 sec
18 20:54:28 SELECT account_id, customer_id, account_type, currency, balance, ... 20 row(s) returned 0.000 sec / 0.000 sec
19 20:54:50 DESCRIBE transactions 10 row(s) returned 0.016 sec / 0.000 sec
20 20:55:25 USE fraud_detection_db 0 row(s) affected 0.000 sec
21 20:55:25 SELECT transaction_id, account_id, transaction_type, amount, merch... 20 row(s) returned 0.016 sec / 0.000 sec

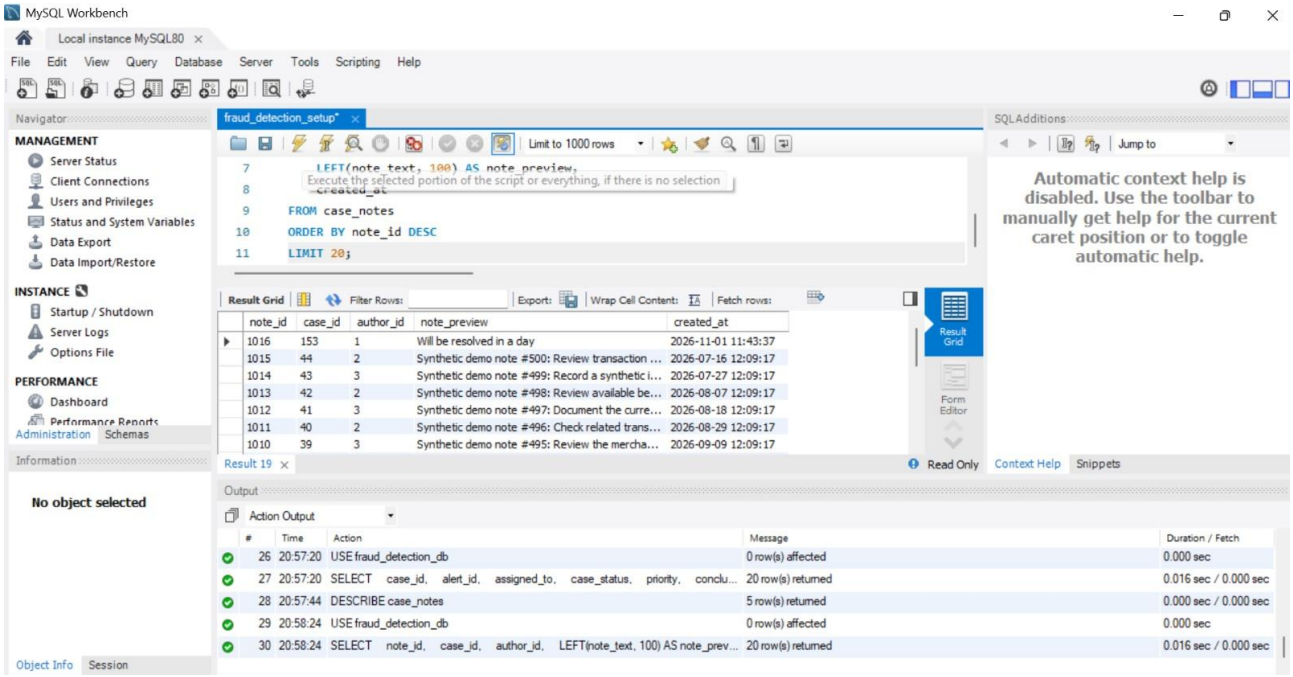
5.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
7 risk_level,
8 alert_status
9 FROM fraud_alerts
10 ORDER BY alert_id DESC
11 LIMIT 20;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch
alert_id transaction_id risk_score risk_level alert_status
303 300 54.00 MEDIUM CONFIRMED_FRAUD
302 299 88.00 HIGH NEW
301 298 51.00 MEDIUM CLOSED
300 297 85.00 HIGH UNDER_REVIEW
299 296 48.00 LOW CLOSED
298 295 82.00 HIGH NEW
297 294 45.00 LOW FALSE_POSITIVE
fraud_alerts 15
Result Grid
Form Editor
Apply
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
20 20:55:25 USE fraud_detection_db 0 row(s) affected 0.000 sec
21 20:55:25 SELECT transaction_id, account_id, transaction_type, amount, merch... 20 row(s) returned 0.016 sec / 0.000 sec
22 20:55:47 DESCRIBE fraud_alerts 9 row(s) returned 0.016 sec / 0.000 sec
23 20:56:23 USE fraud_detection_db 0 row(s) affected 0.000 sec
24 20:56:23 SELECT alert_id, transaction_id, risk_score, risk_level, alert_status FR... 20 row(s) returned 0.000 sec / 0.000 sec

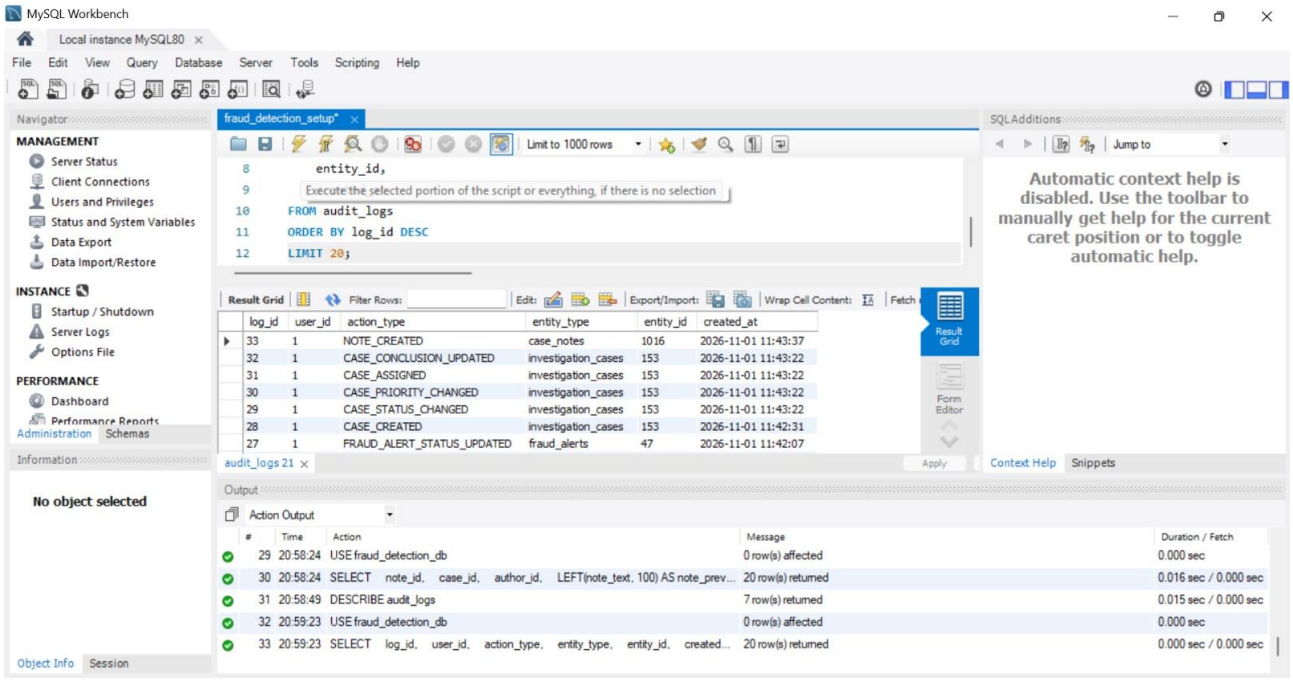
6.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
8 priority,
9 conclusion
10 FROM investigation_cases
11 ORDER BY case_id DESC
12 LIMIT 20;
Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: Fetch
case_id alert_id assigned_to case_status priority conclusion
153 72 2 IN_PROGRESS MEDIUM Fraud level is medium
152 152 2 RESOLVED HIGH Synthetic demo case: investigation completed.
151 151 3 OPEN LOW Synthetic demo case: pending analyst investiga...
150 150 2 OPEN MEDIUM Synthetic demo case: pending analyst investiga...
149 149 3 IN_PROGRESS LOW Synthetic demo case: pending analyst investiga...
148 148 2 OPEN MEDIUM Synthetic demo case: pending analyst investiga...
147 147 3 ESCALATED LOW Synthetic demo case: requires escalation review.
investigation_cases 17
Result Grid
Form Editor
Apply
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
23 20:56:23 USE fraud_detection_db 0 row(s) affected 0.000 sec
24 20:56:23 SELECT alert_id, transaction_id, risk_score, risk_level, alert_status FR... 20 row(s) returned 0.000 sec / 0.000 sec
25 20:56:44 DESCRIBE investigation_cases 8 row(s) returned 0.000 sec / 0.000 sec
26 20:57:20 USE fraud_detection_db 0 row(s) affected 0.000 sec
27 20:57:20 SELECT case_id, alert_id, assigned_to, case_status, priority, conclu... 20 row(s) returned 0.016 sec / 0.000 sec

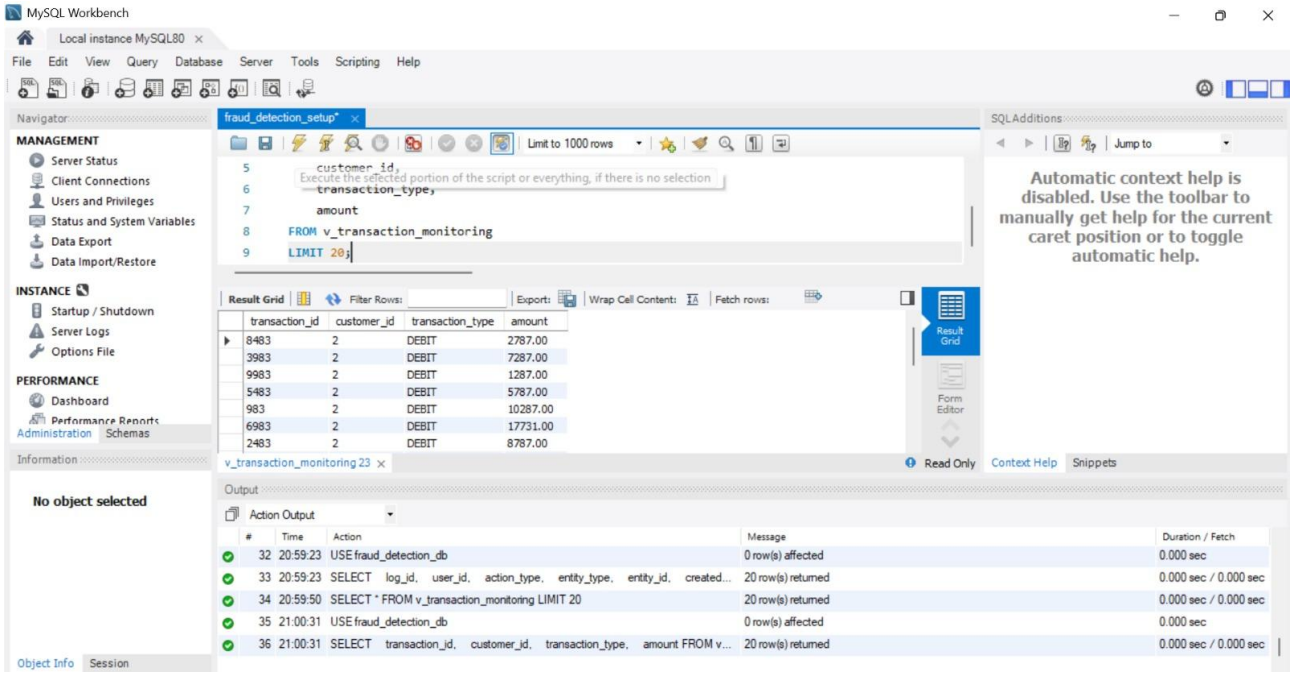
7.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
7 LEFT(note_text, 100) AS note_preview,
8 created_at
Execute the selected portion of the script or everything, if there is no selection
9 FROM case_notes
10 ORDER BY note_id DESC
11 LIMIT 20;
Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:
note_id case_id author_id note_preview created_at
1016 153 1 Will be resolved in a day 2026-11-01 11:43:37
1015 44 2 Synthetic demo note #500: Review transaction ... 2026-07-16 12:09:17
1014 43 3 Synthetic demo note #499: Record a synthetic i... 2026-07-27 12:09:17
1013 42 2 Synthetic demo note #498: Review available be... 2026-08-07 12:09:17
1012 41 3 Synthetic demo note #497: Document the curre... 2026-08-18 12:09:17
1011 40 2 Synthetic demo note #496: Check related trans... 2026-08-29 12:09:17
1010 39 3 Synthetic demo note #495: Review the mercha... 2026-09-09 12:09:17
Result 19
Result Grid
Form Editor
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
26 20:57:20 USE fraud_detection_db 0 row(s) affected 0.000 sec
27 20:57:20 SELECT case_id, alert_id, assigned_to, case_status, priority, conclu... 20 row(s) returned 0.016 sec / 0.000 sec
28 20:57:44 DESCRIBE case_notes 5 row(s) returned 0.000 sec / 0.000 sec
29 20:58:24 USE fraud_detection_db 0 row(s) affected 0.000 sec
30 20:58:24 SELECT note_id, case_id, author_id, LEFT(note_text, 100) AS note_prev... 20 row(s) returned 0.016 sec / 0.000 sec

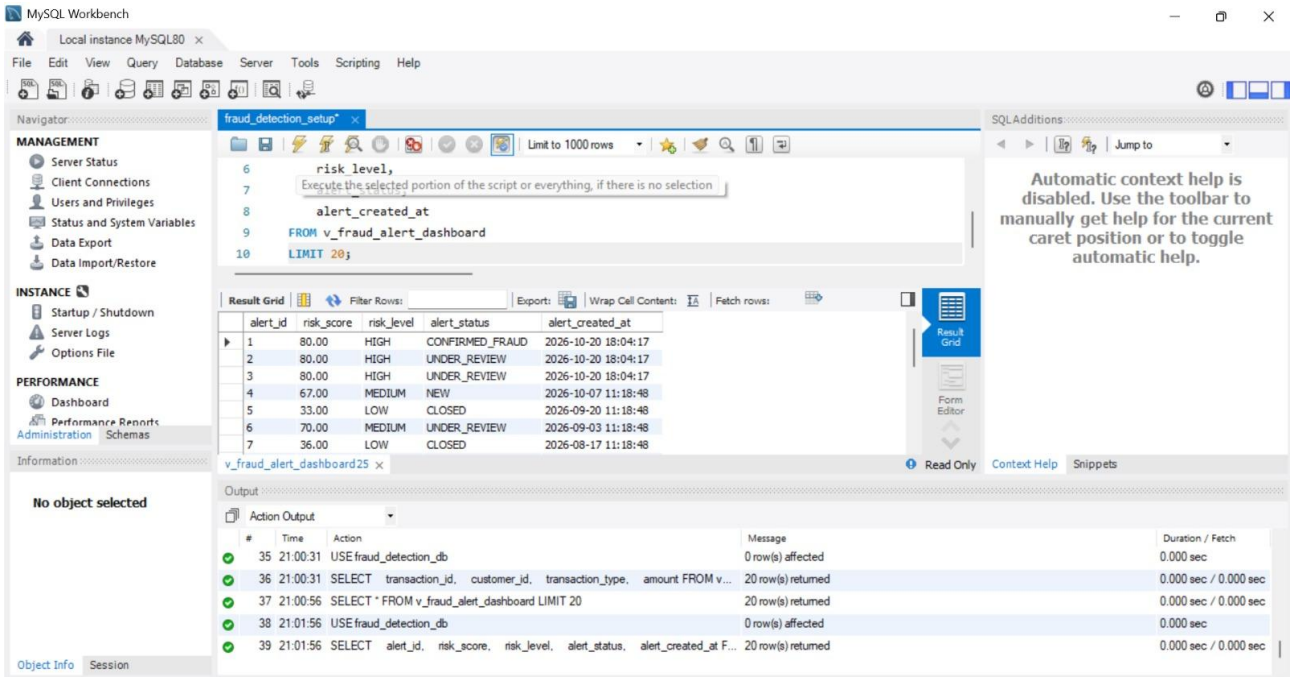
8.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
8 entity_id,
9 Execute the selected portion of the script or everything, if there is no selection
10 FROM audit_logs
11 ORDER BY log_id DESC
12 LIMIT 20;
Result Grid
log_id user_id action_type entity_type entity_id created_at
33 1 NOTE_CREATED case_notes 1016 2026-11-01 11:43:37
32 1 CASE_CONCLUSION_UPDATED investigation_cases 153 2026-11-01 11:43:22
31 1 CASE_ASSIGNED investigation_cases 153 2026-11-01 11:43:22
30 1 CASE_PRIORITY_CHANGED investigation_cases 153 2026-11-01 11:43:22
29 1 CASE_STATUS_CHANGED investigation_cases 153 2026-11-01 11:43:22
28 1 CASE_CREATED investigation_cases 153 2026-11-01 11:42:31
27 1 FRAUD_ALERT_STATUS_UPDATED fraud_alerts 47 2026-11-01 11:42:07
audit_logs 21
SQL Additions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
29 20:58:24 USE fraud_detection_db 0 row(s) affected 0.000 sec
30 20:58:24 SELECT note_id, case_id, author_id, LEFT(note_text, 100) AS note_prev... 20 row(s) returned 0.016 sec / 0.000 sec
31 20:58:49 DESCRIBE audit_logs 7 row(s) returned 0.015 sec / 0.000 sec
32 20:59:23 USE fraud_detection_db 0 row(s) affected 0.000 sec
33 20:59:23 SELECT log_id, user_id, action_type, entity_type, entity_id, created... 20 row(s) returned 0.000 sec / 0.000 sec

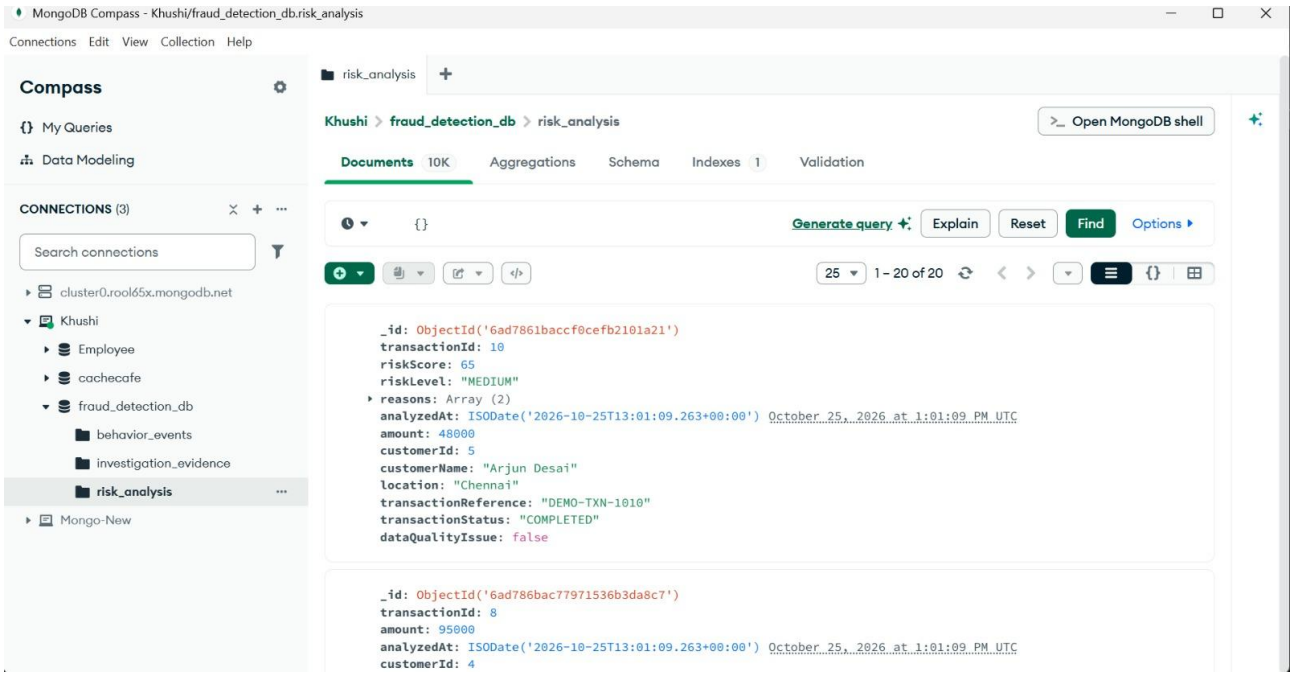
9.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
5 customer_id,
6 transaction_type,
7 amount
8 FROM v_transaction_monitoring
9 LIMIT 20;
Execute the selected portion of the script or everything, if there is no selection
Result Grid
transaction_id customer_id transaction_type amount
8483 2 DEBIT 2787.00
3983 2 DEBIT 7287.00
9983 2 DEBIT 1287.00
5483 2 DEBIT 5787.00
983 2 DEBIT 10287.00
6983 2 DEBIT 17731.00
2483 2 DEBIT 8787.00
v_transaction_monitoring 23
Read Only
SQL Additions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
32 20:59:23 USE fraud_detection_db 0 row(s) affected 0.000 sec
33 20:59:23 SELECT log_id, user_id, action_type, entity_type, entity_id, created... 20 row(s) returned 0.000 sec / 0.000 sec
34 20:59:50 SELECT * FROM v_transaction_monitoring LIMIT 20 20 row(s) returned 0.000 sec / 0.000 sec
35 21:00:31 USE fraud_detection_db 0 row(s) affected 0.000 sec
36 21:00:31 SELECT transaction_id, customer_id, transaction_type, amount FROM v... 20 row(s) returned 0.000 sec / 0.000 sec

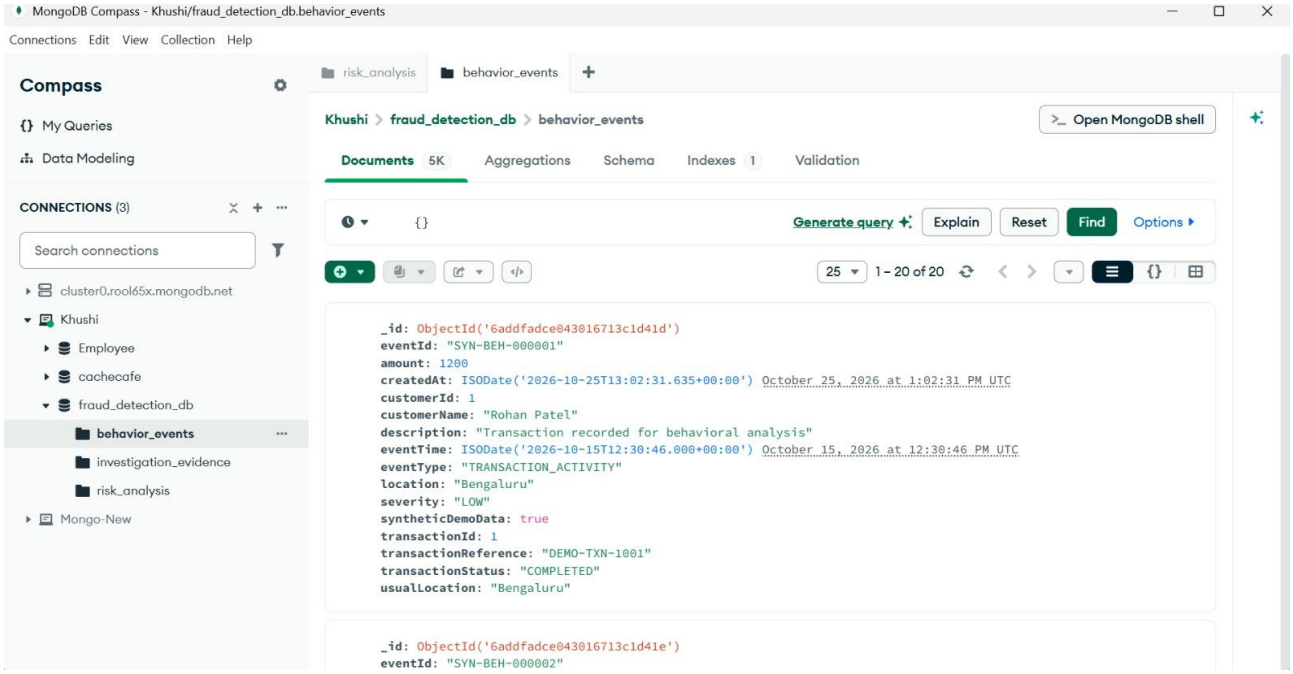
10.


MySQL Workbench
Local instance MySQL80
File Edit View Query Database Server Tools Scripting Help
Navigator
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Administration Schemas
Information
No object selected
Object Info Session
fraud_detection_setup*
Limit to 1000 rows
risk_level,
Execute the selected portion of the script or everything, if there is no selection
alert_created_at
FROM v_fraud_alert_dashboard
LIMIT 20;
Result Grid Filter Rows: Export: Wrap Cell Content: Fetch rows:
alert_id risk_score risk_level alert_status alert_created_at
1 80.00 HIGH CONFIRMED_FRAUD 2026-10-20 18:04:17
2 80.00 HIGH UNDER_REVIEW 2026-10-20 18:04:17
3 80.00 HIGH UNDER_REVIEW 2026-10-20 18:04:17
4 67.00 MEDIUM NEW 2026-10-07 11:18:48
5 33.00 LOW CLOSED 2026-09-20 11:18:48
6 70.00 MEDIUM UNDER_REVIEW 2026-09-03 11:18:48
7 36.00 LOW CLOSED 2026-08-17 11:18:48
v_fraud_alert_dashboard25
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
35 21:00:31 USE fraud_detection_db 0 row(s) affected 0.000 sec
36 21:00:31 SELECT transaction_id, customer_id, transaction_type, amount FROM v... 20 row(s) returned 0.000 sec / 0.000 sec
37 21:00:56 SELECT * FROM v_fraud_alert_dashboard LIMIT 20 20 row(s) returned 0.000 sec / 0.000 sec
38 21:01:56 USE fraud_detection_db 0 row(s) affected 0.000 sec
39 21:01:56 SELECT alert_id, risk_score, risk_level, alert_status, alert_created_at F... 20 row(s) returned 0.000 sec / 0.000 sec

11.


MongoDB Compass - Khushi/fraud_detection_db.risk_analysis
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (3)
Search connections
cluster0.rool65x.mongodb.net
Khushi
Employee
cachecafe
fraud_detection_db
behavior_events
investigation_evidence
risk_analysis
Mongo-New
risk_analysis
Khushi > fraud_detection_db > risk_analysis
Open MongoDB shell
Documents 10K Aggregations Schema Indexes 1 Validation
Generate query Explain Reset Find Options
25 1 - 20 of 20
_id: ObjectId('6ad7861baccf0cefb2101a21')
transactionId: 10
riskScore: 65
riskLevel: "MEDIUM"
reasons: Array (2)
analyzedAt: ISODate('2026-10-25T13:01:09.263+00:00') October 25, 2026 at 1:01:09 PM UTC
amount: 48000
customerId: 5
customerName: "Arjun Desai"
location: "Chennai"
transactionReference: "DEMO-TXN-1010"
transactionStatus: "COMPLETED"
dataQualityIssue: false
_id: ObjectId('6ad786bac77971536b3da8c7')
transactionId: 8
amount: 95000
analyzedAt: ISODate('2026-10-25T13:01:09.263+00:00') October 25, 2026 at 1:01:09 PM UTC
customerId: 4

12.



Summary

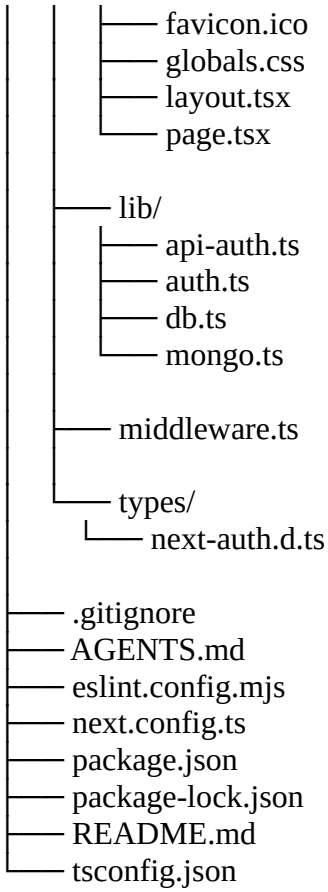
The MongoDB implementation demonstrates document storage, bulk write operations, update operations with upsert, document counting, and aggregation pipelines.
These features support the management of flexible analytical information and complement the structured relational database maintained using MySQL.

9. Next.js Project Tree Structure

The SentinelAI application is developed using Next.js with the App Router architecture. The project follows a modular structure in which the frontend pages, backend API routes, database connectivity modules, authentication logic and supporting scripts are maintained separately.

Project Directory Structure:

```
fraud-detection-system/
├── public/
│   ├── file.svg
│   ├── globe.svg
│   ├── next.svg
│   ├── vercel.svg
│   └── window.svg
├── scripts/
│   └── seed-mongodb.cjs
├── src/
│   ├── app/
│   │   ├── api/
│   │   │   ├── audit-logs/
│   │   │   │   └── route.ts
│   │   │   ├── auth/
│   │   │   │   └── [...nextauth]/
│   │   │   │       └── route.ts
│   │   │   ├── db-test/
│   │   │   │   └── route.ts
│   │   │   ├── fraud-alerts/
│   │   │   │   └── route.ts
│   │   │   ├── investigations/
│   │   │   │   ├── notes/
│   │   │   │   │   └── route.ts
│   │   │   │   └── route.ts
│   │   │   ├── mongo-test/
│   │   │   │   └── route.ts
│   │   │   ├── risk-analysis/
│   │   │   │   └── route.ts
│   │   │   ├── transaction-audit/
│   │   │   │   └── route.ts
│   │   │   ├── transactions/
│   │   │   │   └── route.ts
│   │   │   └── users/
│   │   │       └── route.ts
│   │   ├── login/
│   │   │   └── page.tsx
```



Description of Major Directories and Files:

1. **public/** – Contains static assets such as SVG icons used by the application.
2. **scripts/** – Contains the MongoDB seeding script used to populate the MongoDB collections with transaction risk analysis, behavioral events and investigation evidence.
3. **src/app/** – Contains the frontend pages and backend API routes. The main dashboard is implemented in page.tsx, while login/page.tsx handles the login interface.
4. **src/app/api/** – Contains API endpoints for managing transactions, fraud alerts, investigations, investigation notes, risk analysis, audit logs and user-related operations. It also includes authentication and database connectivity test endpoints.
5. **src/lib/** – Contains reusable modules for MySQL connectivity, MongoDB connectivity, authentication and API authorization.
6. **src/types/** – Contains TypeScript type declarations used by the authentication implementation.
7. **middleware.ts** – Implements middleware-level request handling and access control.
8. **package.json** – Defines project dependencies and application scripts.
9. **next.config.ts** – Contains Next.js configuration.
10. **tsconfig.json** – Contains TypeScript compiler configuration.

Summary:

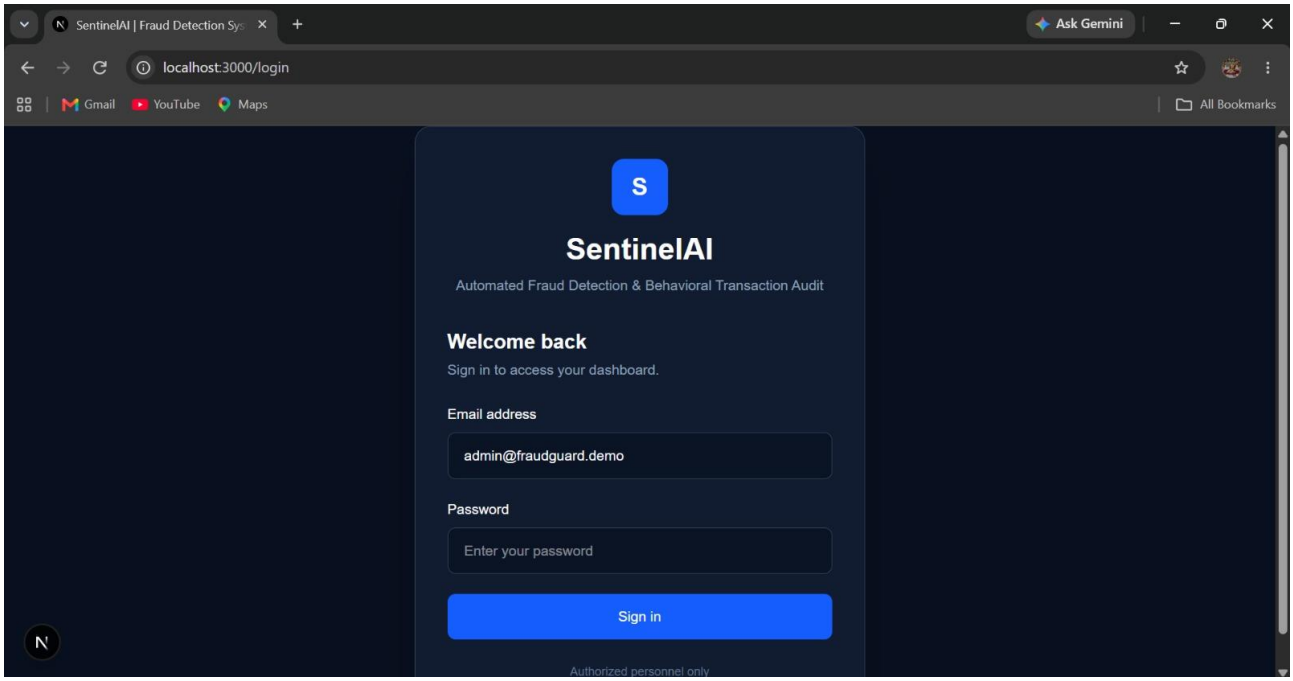
The project structure separates frontend development, backend API functionality, database integration and authentication logic into dedicated modules. This modular organization improves maintainability, code readability and the overall organization of the SentinelAI application.

10. Sample Screenshots of Application Code and Demonstration

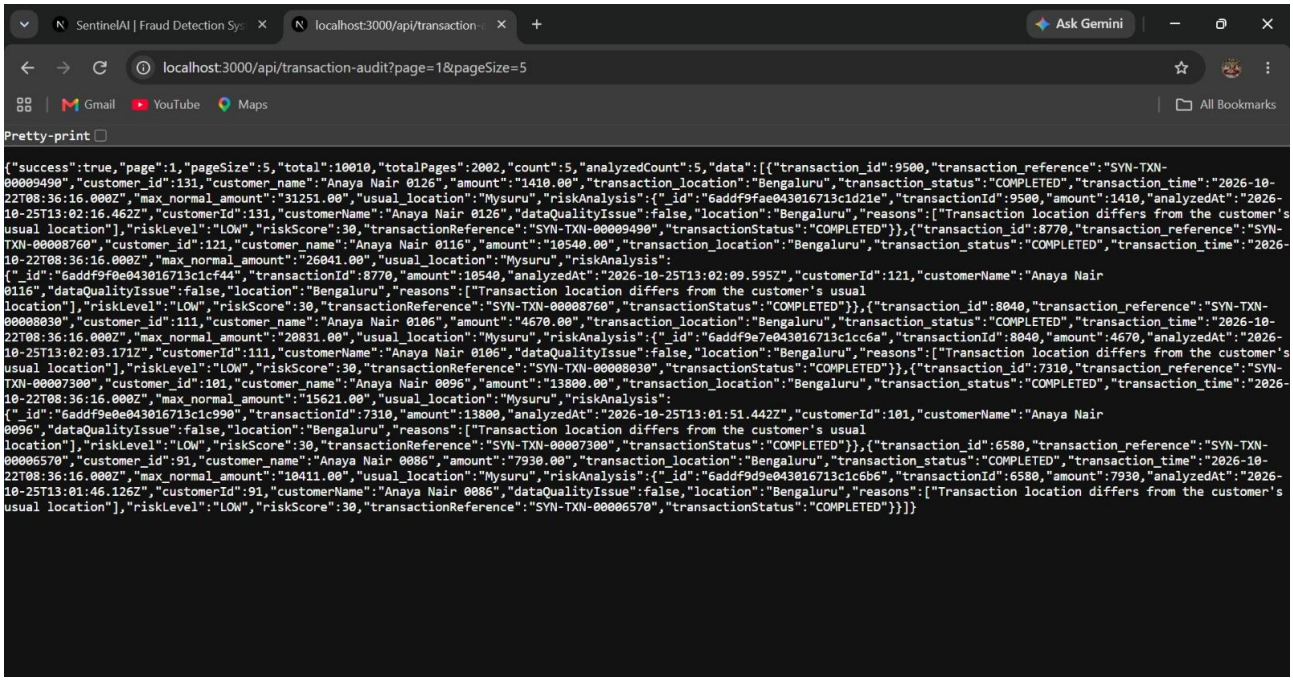
This section presents selected screenshots of the SentinelAI application, demonstrating its frontend implementation, backend API integration, database connectivity and major application interfaces. The screenshots illustrate the implementation of user authentication, transaction monitoring, fraud alert management, investigation handling and integration with MySQL and MongoDB databases. The application is developed using Next.js and TypeScript, following a modular architecture that separates frontend components, API routes and database connectivity modules.

The following screenshots provide evidence of the application's implementation and demonstrate its major functionalities.

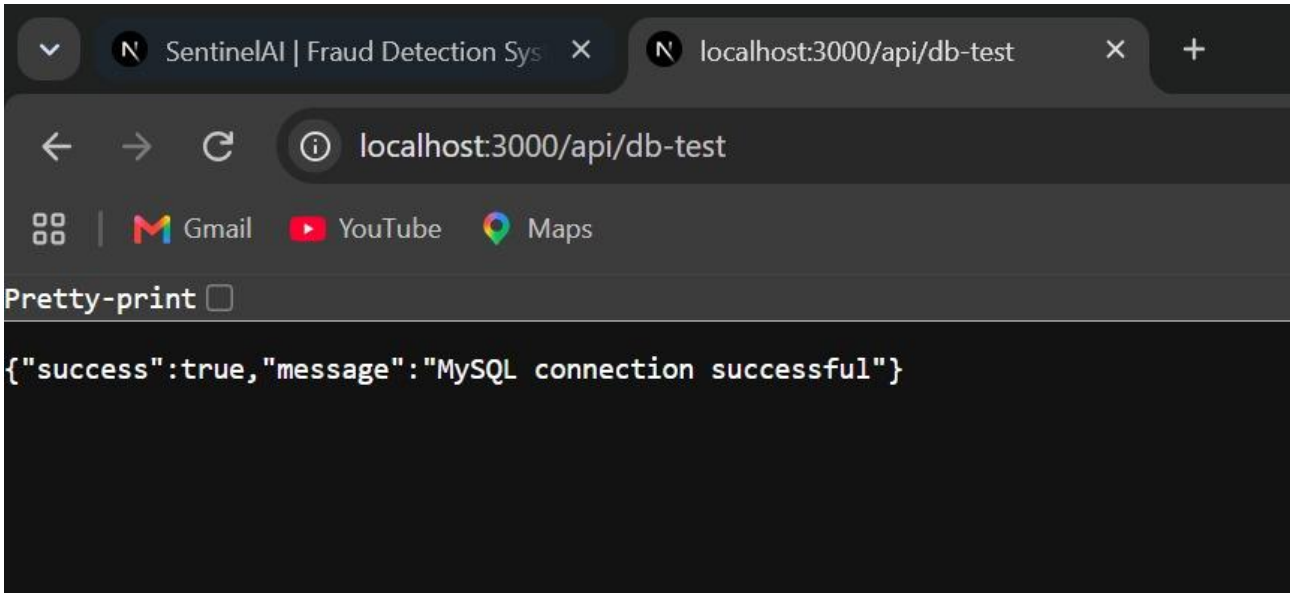
1.



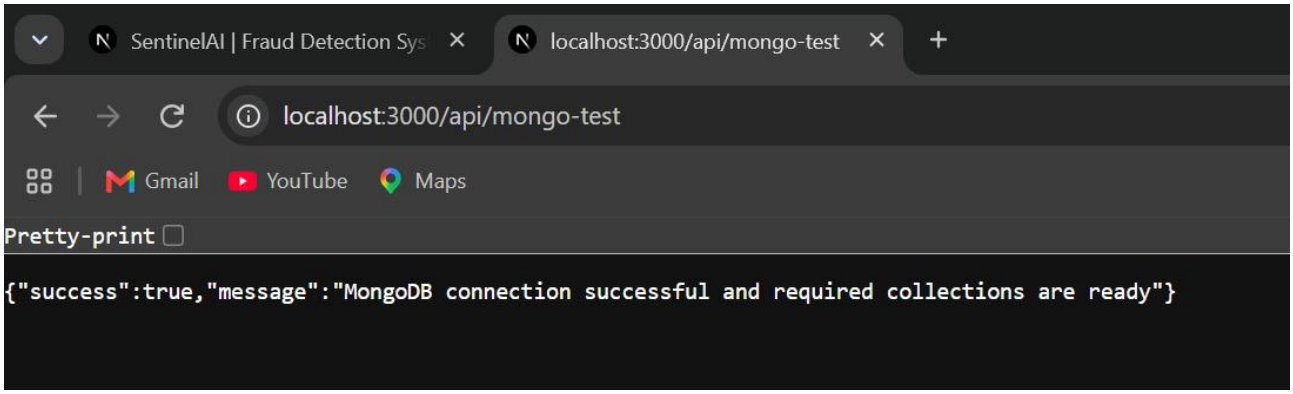
2.


SentinelAI | Fraud Detection Sys
localhost:3000/api/transaction
Ask Gemini
localhost:3000/api/transaction-audit?page=1&pageSize=5
Gmail
YouTube
Maps
All Bookmarks
Pretty-print
{"success":true,"page":1,"pageSize":5,"total":10010,"totalPages":2002,"count":5,"analyzedCount":5,"data":[{"transaction_id":9500,"transaction_reference":"SYN-TXN-00009490","customer_id":131,"customer_name":"Anaya Nair 0126","amount":"1410.00","transaction_location":"Bengaluru","transaction_status":"COMPLETED","transaction_time":"2026-10-22T08:36:16.000Z","max_normal_amount":"31251.00","usual_location":"Mysuru","riskAnalysis":{"_id":"6addf9fae043016713c1d21e","transactionId":9500,"amount":1410,"analyzedAt":"2026-10-25T13:02:16.462Z","customerId":131,"customerName":"Anaya Nair 0126","dataQualityIssue":false,"location":"Bengaluru","reasons":["Transaction location differs from the customer's usual location"],"riskLevel":"LOW","riskScore":30,"transactionReference":"SYN-TXN-00009490","transactionStatus":"COMPLETED"}},{"transaction_id":8770,"transaction_reference":"SYN-TXN-00008760","customer_id":121,"customer_name":"Anaya Nair 0116","amount":"10540.00","transaction_location":"Bengaluru","transaction_status":"COMPLETED","transaction_time":"2026-10-22T08:36:16.000Z","max_normal_amount":"26041.00","usual_location":"Mysuru","riskAnalysis":
{"_id":"6addf9f0e043016713c1cf44","transactionId":8770,"amount":10540,"analyzedAt":"2026-10-25T13:02:09.595Z","customerId":121,"customerName":"Anaya Nair 0116","dataQualityIssue":false,"location":"Bengaluru","reasons":["Transaction location differs from the customer's usual location"],"riskLevel":"LOW","riskScore":30,"transactionReference":"SYN-TXN-00008760","transactionStatus":"COMPLETED"}},{"transaction_id":8040,"transaction_reference":"SYN-TXN-00008030","customer_id":111,"customer_name":"Anaya Nair 0106","amount":"4670.00","transaction_location":"Bengaluru","transaction_status":"COMPLETED","transaction_time":"2026-10-22T08:36:16.000Z","max_normal_amount":"20831.00","usual_location":"Mysuru","riskAnalysis":{"_id":"6addf9e7e043016713c1cc6a","transactionId":8040,"amount":4670,"analyzedAt":"2026-10-25T13:02:03.171Z","customerId":111,"customerName":"Anaya Nair 0106","dataQualityIssue":false,"location":"Bengaluru","reasons":["Transaction location differs from the customer's usual location"],"riskLevel":"LOW","riskScore":30,"transactionReference":"SYN-TXN-00008030","transactionStatus":"COMPLETED"}},{"transaction_id":7310,"transaction_reference":"SYN-TXN-00007300","customer_id":101,"customer_name":"Anaya Nair 0096","amount":"13800.00","transaction_location":"Bengaluru","transaction_status":"COMPLETED","transaction_time":"2026-10-22T08:36:16.000Z","max_normal_amount":"15621.00","usual_location":"Mysuru","riskAnalysis":
{"_id":"6addf9e0e043016713c1c990","transactionId":7310,"amount":13800,"analyzedAt":"2026-10-25T13:01:51.442Z","customerId":101,"customerName":"Anaya Nair 0096","dataQualityIssue":false,"location":"Bengaluru","reasons":["Transaction location differs from the customer's usual location"],"riskLevel":"LOW","riskScore":30,"transactionReference":"SYN-TXN-00007300","transactionStatus":"COMPLETED"}},{"transaction_id":6580,"transaction_reference":"SYN-TXN-00006570","customer_id":91,"customer_name":"Anaya Nair 0086","amount":"7930.00","transaction_location":"Bengaluru","transaction_status":"COMPLETED","transaction_time":"2026-10-22T08:36:16.000Z","max_normal_amount":"10411.00","usual_location":"Mysuru","riskAnalysis":{"_id":"6addf9d9e043016713c1c6b6","transactionId":6580,"amount":7930,"analyzedAt":"2026-10-25T13:01:46.126Z","customerId":91,"customerName":"Anaya Nair 0086","dataQualityIssue":false,"location":"Bengaluru","reasons":["Transaction location differs from the customer's usual location"],"riskLevel":"LOW","riskScore":30,"transactionReference":"SYN-TXN-00006570","transactionStatus":"COMPLETED"}}]}

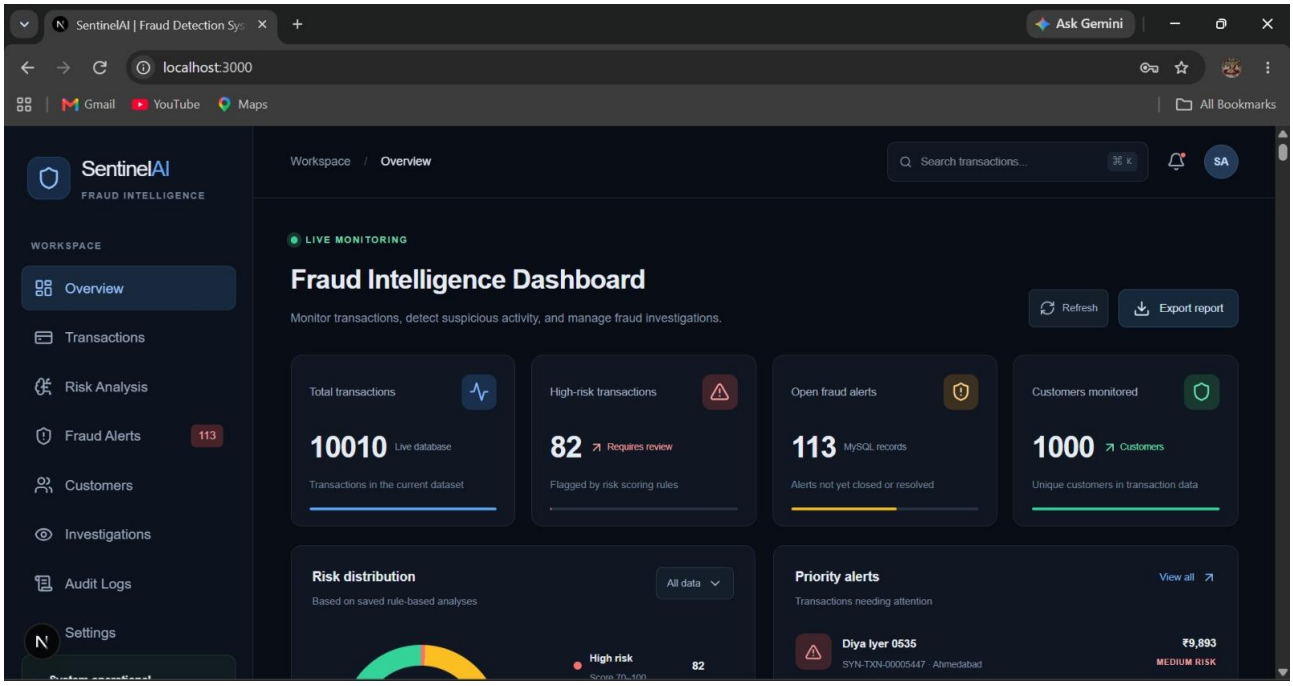
3.


SentinelAI | Fraud Detection Sys
localhost:3000/api/db-test
localhost:3000/api/db-test
Gmail
YouTube
Maps
Pretty-print
{"success":true,"message":"MySQL connection successful"}

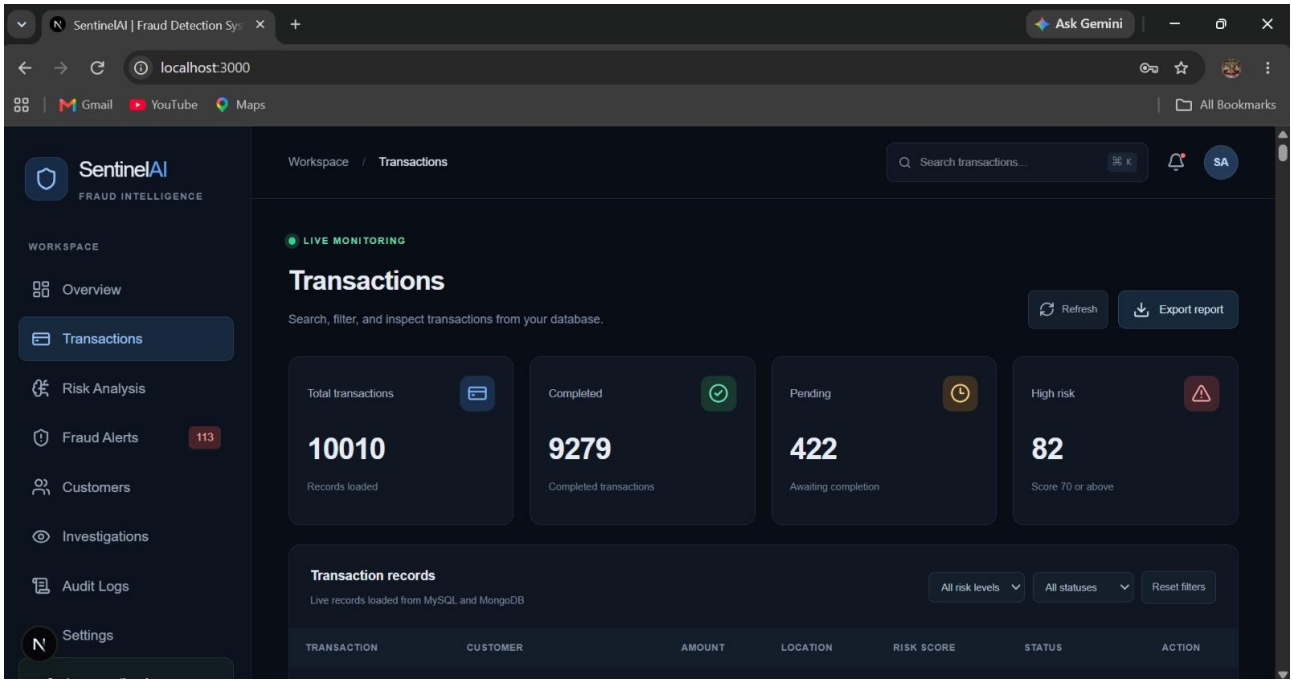
4.


SentinelAI | Fraud Detection Sys
localhost:3000/api/mongo-test
localhost:3000/api/mongo-test
Gmail
YouTube
Maps
Pretty-print
{"success":true,"message":"MongoDB connection successful and required collections are ready"}

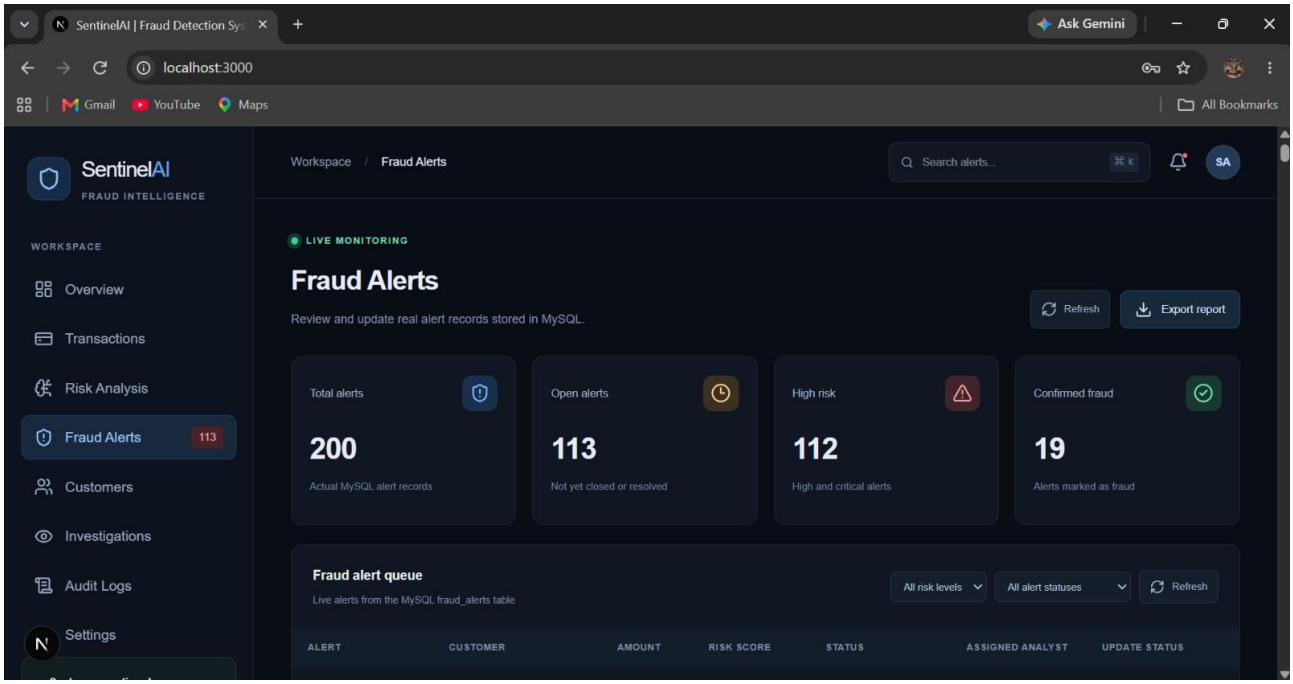
5.


SentinelAI | Fraud Detection Sys
Ask Gemini
localhost:3000
Gmail
YouTube
Maps
All Bookmarks
SentinelAI
FRAUD INTELLIGENCE
Workspace / Overview
Search transactions...
SA
WORKSPACE
Overview
Transactions
Risk Analysis
Fraud Alerts 113
Customers
Investigations
Audit Logs
Settings
LIVE MONITORING
Fraud Intelligence Dashboard
Monitor transactions, detect suspicious activity, and manage fraud investigations.
Refresh
Export report
Total transactions
10010 Live database
Transactions in the current dataset
High-risk transactions
82 Requires review
Flagged by risk scoring rules
Open fraud alerts
113 MySQL records
Alerts not yet closed or resolved
Customers monitored
1000 Customers
Unique customers in transaction data
Risk distribution
Based on saved rule-based analyses
All data
High risk
82
Priority alerts
Transactions needing attention
View all
Diya Iyer 0535
SYN-TXN-00005447 · Ahmedabad
₹9,893
MEDIUM RISK

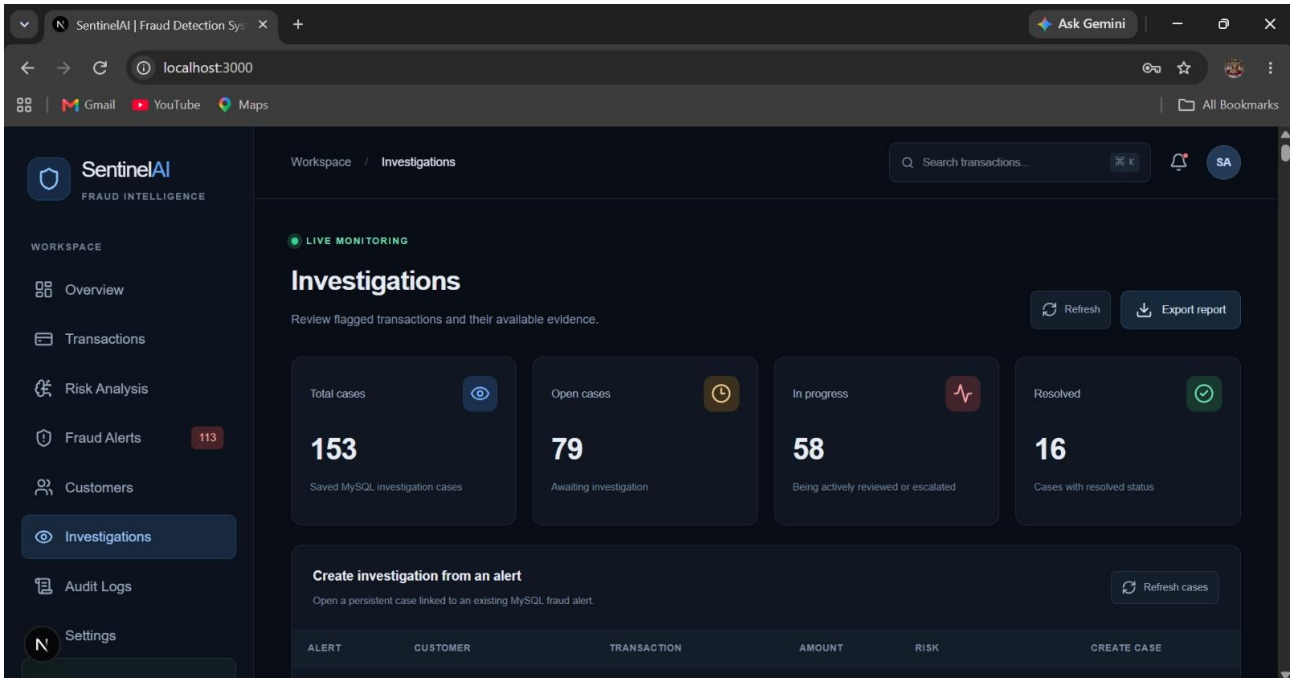
6.


SentinelAI | Fraud Detection Sys
Ask Gemini
localhost:3000
Gmail
YouTube
Maps
All Bookmarks
SentinelAI
FRAUD INTELLIGENCE
Workspace / Transactions
Search transactions...
SA
WORKSPACE
Overview
Transactions
Risk Analysis
Fraud Alerts 113
Customers
Investigations
Audit Logs
Settings
LIVE MONITORING
Transactions
Search, filter, and inspect transactions from your database.
Refresh
Export report
Total transactions
10010
Records loaded
Completed
9279
Completed transactions
Pending
422
Awaiting completion
High risk
82
Score 70 or above
Transaction records
Live records loaded from MySQL and MongoDB
All risk levels
All statuses
Reset filters
TRANSACTION
CUSTOMER
AMOUNT
LOCATION
RISK SCORE
STATUS
ACTION

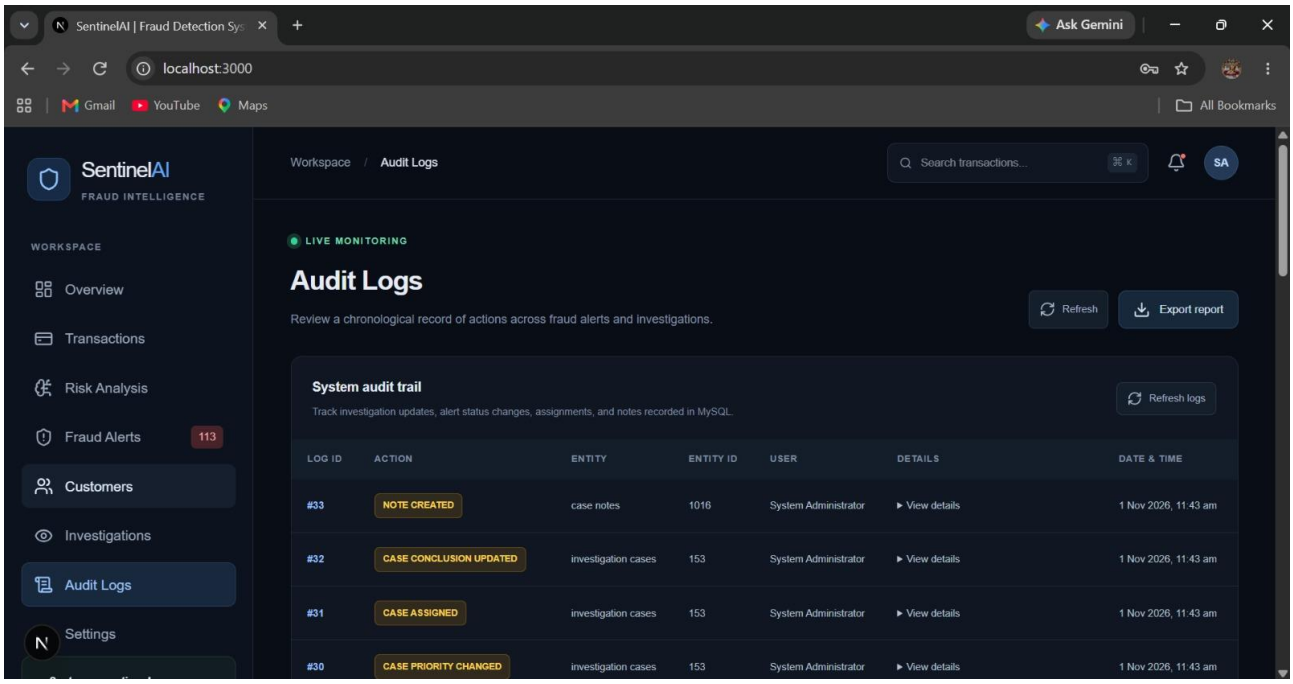
7.


SentinelAI | Fraud Detection Sys
Ask Gemini
localhost:3000
Gmail
YouTube
Maps
All Bookmarks
SentinelAI
FRAUD INTELLIGENCE
Workspace / Fraud Alerts
Search alerts...
SA
WORKSPACE
Overview
Transactions
Risk Analysis
Fraud Alerts 113
Customers
Investigations
Audit Logs
Settings
LIVE MONITORING
Fraud Alerts
Review and update real alert records stored in MySQL.
Refresh
Export report
Total alerts
200
Actual MySQL alert records
Open alerts
113
Not yet closed or resolved
High risk
112
High and critical alerts
Confirmed fraud
19
Alerts marked as fraud
Fraud alert queue
Live alerts from the MySQL fraud_alerts table
All risk levels
All alert statuses
Refresh
ALERT
CUSTOMER
AMOUNT
RISK SCORE
STATUS
ASSIGNED ANALYST
UPDATE STATUS

8.



9.



Conclusion:

SentinelAI successfully integrates a Next.js-based web application with MySQL and MongoDB to support transaction monitoring, rule-based risk assessment, fraud alert management and investigation tracking. The major application modules have been implemented, and the final

database verification, screenshot collection and live demonstration will be completed as part of the final evaluation.

Evaluation Rubrics

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	completed
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	completed
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	completed
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	completed
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	completed
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	completed
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	completed